

OpenSTA

Table of Contents

Command Line Arguments.....	1
Example Command Scripts.....	1
Timing Analysis using SDF.....	2
Timing Analysis with Multiple Process Corners.....	2
Timing Analysis with Multiple Corners and Modes.....	3
Power Analysis.....	3
TCL Interpreter.....	5
Debugging Timing.....	6
No paths found.....	6
No path reported an endpoint.....	7
Commands.....	7
Filter Expressions.....	84
Variables.....	85

Command Line Arguments

The command line arguments for `sta` are shown below.

```
sta
  -help                show help and exit
  -version             show version and exit
  -no_init             do not read ~/.sta
  -no_splash           do not print the splash message
  -threads count|max  use count threads
  -exit               exit after reading cmd_file
  cmd_file           source cmd_file
```

When OpenSTA starts up, commands are first read from the user initialization file `~/.sta` if it exists. If a TCL command file *cmd_file* is specified on the command line, commands are read from the file and executed before entering an interactive TCL command interpreter. If `-exit` is specified the application exits after reading *cmd_file*. Use the TCL `exit` command to exit the application. The `-threads` option specifies how many parallel threads to use. Use `-threads max` to use one thread per processor.

Example Command Scripts

To read a design into OpenSTA use the `read_liberty` command to read Liberty library files. Next, read hierarchical structural Verilog files with the `read_verilog` command. The `link_design` command links the Verilog to the Liberty timing cells. Any number of Liberty and Verilog files can be read before linking the design.

Delays used for timing analysis are calculated using the Liberty timing models. If no parasitics are read only the pin capacitances of the timing models are used in delay calculation. Use the `read_spef` command to read parasitics from an extractor, or `read_sdf` to use delays calculated by an external delay calculator.

Timing constraints can be entered as TCL commands or read using the `read_sdc` command.

The units used by OpenSTA for all command arguments and reports are taken from the first Liberty file that is read. Use the `set_cmd_units` command to override the default units. Use the `report_units` command to see the command units.

Timing Analysis using SDF

A sample command file that reads a library and a Verilog netlist and reports timing checks is shown below.

```
read_liberty example1_slow.lib
read_verilog example1.v
link_design top
read_sdf example1.sdf
create_clock -name clk -period 10 {clk1 clk2 clk3}
set_input_delay -clock clk 0 {in1 in2}
report_checks
```

This example can be found in `examples/sdf_delays.tcl`.

Timing Analysis with Multiple Process Corners

An example command script using three process corners and +/-10% min/max derating is shown below.

```
read_liberty nangate45_slow.lib.gz
read_liberty nangate45_typ.lib.gz
read_liberty nangate45_fast.lib.gz
read_verilog example1.
link_design top
set_timing_derate -early 0.9
set_timing_derate -late 1.1
create_clock -name clk -period 10 {clk1 clk2 clk3}
set_input_delay -clock clk 0 {in1 in2}

define_scene ss -liberty nangate45_slow
define_scene tt -liberty nangate45_typ
define_scene ff -liberty nangate45_fast

# report all scenes
report_checks -path_delay min_max
# report typical scene
report_checks -scene tt
```

This example can be found in `examples/multi_corner.tcl`. Other examples can be found in the `examples` directory.

Timing Analysis with Multiple Corners and Modes

OpenSTA supports multi-corner, multi-mode analysis. Each corner/mode combination is called a “scene”. The SDC constraints in each mode describe a different operating mode, such as mission mode or scan mode. Each corner has min/max Liberty libraries and SPEF parasitics.

A mode named “default” is initially created for SDC commands. It is deleted when a mode is defined with `set_mode` or `read_sdc -mode`. Similarly, a named “default” is initially created that is deleted when `define_scene` is used to define a scene.

An example command script using two process corners two modes is shown below.

```
read_liberty asap7_small_ff.lib.gz
read_liberty asap7_small_ss.lib.gz
read_verilog reg1_asap7.v
link_design top

read_sdc -mode model1 mcmm2_model1.sdc
read_sdc -mode mode2 mcmm2_mode2.sdc

read_spef -name reg1_ff reg1_asap7.spef
read_spef -name reg1_ss reg1_asap7_ss.spef

define_scene scenel -mode model1 -liberty asap7_small_ff -spef reg1_ff
define_scene scene2 -mode mode2 -liberty asap7_small_ss -spef reg1_ss

report_checks -scenes scenel
report_checks -scenes scene2
report_checks -group_path_count 4
```

This example can be found in `examples/mcmm3.tcl`.

In the example show above the SDC for each mode is in defined in a separate file. Alternatively, the SDC can be defined in the command file using the `set_mode` command between SDC command groups.

```

set_mode mode1
create_clock -name m1_clk -period 1000 {clk1 clk2 clk3}
set_input_delay -clock m1_clk 100 {in1 in2}

set_mode mode2
create_clock -name m2_clk -period 500 {clk1 clk3}
set_output_delay -clock m2_clk 100 out

```

Analysis Corners

An analysis corner is a named collection of the setup data that varies with an operating point: the min/max Liberty libraries, the SPEF parasitics, and the SDC constraints that describe that operating point. A scene is one mode paired with one analysis corner, so corners are defined once and reused across modes.

```

define_analysis_corner ss_cmax -liberty asap7_small_ss -spef reg1_ss -sdc
derates_ss.sdc
read_sdc -mode func func.sdc
define_scene func_ss -mode func -analysis_corner ss_cmax

```

The corner's Liberty and SPEF arguments are composed into `define_scene`, so the resulting scene is the same as one defined with explicit `-liberty` and `-spef` arguments; explicit arguments take precedence over the corner's. The corner's SDC files are read for a mode and corner combination when the first scene naming that pair is defined. Use `-liberty_min/-liberty_max` and `-spef_min/-spef_max` for different min and max data; they are mutually exclusive with `-liberty` and `-spef`.

Constraints that describe an operating point can be scoped to a corner: timing derates, input and output delays, clock uncertainty, and clock latency and source insertion. Structural constraints such as clocks, exceptions and case analysis are mode level and are rejected in a corner scope. A corner scope is entered with `read_sdc -analysis_corner` or interactively with `set_cmd_analysis_corner`.

```

read_sdc -mode func -analysis_corner ss_cmax derates_ss.sdc

set_cmd_analysis_corner ss_cmax
set_timing_derate -late 1.1
unset_cmd_analysis_corner

```

Corner constraints override the mode's for the scenes that use the corner, and the mode's constraints apply where the corner defines none. Analysis corners are objects: `get_analysis_corners` finds them by pattern or with `-filter`, `get_property` reads their properties, `define_property` and `set_property` with `-object_type analysis_corner` add user defined properties, and `get_scenes -filter {analysis_corner == ss_cmax}` selects the scenes using a corner. `set_scene_analysis_corner` associates an existing scene with a corner.

Statistical Timing Analysis

OpenSTA also supports statistical timing analysis with Liberty Variation Format (LVF) libraries. Statistical timing uses a probability distribution to represent a delay or slew rather than a single number.

Normal and skew normal probability distributions are supported. SSTA is enabled with the `sta_pocv_mode` variable.

```
set sta_pocv_mode scalar|normal|skew_normal
```

```
scalar mode is for non-SSTA analysis
```

```
normal mode uses gaussian normal distributions
```

```
skew_normal mode is for skew normal LVF moment based distributions
```

The target quantile of a delay probability distribution (confidence level) is set with the `sta_pocv_quantile` variable.

```
set sta_pocv_quantile <float>
```

The default value is 3 standard deviations, or sigma.

Use the `variation` field with the `report_checks` and `report_check_types` commands to see distribution parameters in timing reports.

A command file for analyzing a design with statistical timing is shown below.

```
read_liberty lvf_library.lib.gz
read_verilog design.v
link_design topcreate_clock -period 50 clk
set_input_delay -clock clk 1 {in1 in2}
set_sta_pocv_mode skew_normal
report_checks -fields {slew variation input_pin} -digits 3
```

Startpoint: r2 (rising edge-triggered flip-flop clocked by clk)
 Endpoint: r3 (rising edge-triggered flip-flop clocked by clk)
 Path Group: clk
 Path Type: max

Slew	Delay Variation	Time	Description
0.000	0.000	0.000	clock clk (rise edge)
	0.000	0.000	clock network delay (ideal)
0.000	0.000	0.000	^ r2/CK (FDPQ1)
	12.026		mean
	0.017		mean_shift
	0.366		std_dev
	0.000		skewness
4.648	12.409	12.409	v r2/Q (FFQ1)
4.648	0.000	12.409	v u1/A (BUF1)
	6.084		mean
	0.007		mean_shift
	0.188		std_dev
	0.000		skewness
2.513	6.137	18.546	v u1/X (BUF1)
2.513	0.000	18.546	v u2/A2 (AN21)
	6.447		mean
	0.008		mean_shift
	0.191		std_dev
	0.000		skewness
2.565	6.497	25.043	v u2/X (AN21)
2.565	0.000	25.043	v r3/D (FFQ1)
		25.043	data arrival time
0.000	50.000	50.000	clock clk (rise edge)
	0.000	50.000	clock network delay (ideal)
	0.000	50.000	clock reconvergence pessimism
		50.000	^ r3/CK (FFQ1)
	-9.376	40.624	library setup time
		40.624	data required time
		40.624	data required time
		-25.043	data arrival time
		15.581	slack (MET)

The standard deviation for normal distributions is specified with the following liberty timing groups.

```
ocv_sigma_cell_rise
ocv_sigma_cell_fall
ocv_sigma_rise_transition
ocv_sigma_fall_transition
ocv_sigma_rise_constraint
ocv_sigma_fall_constraint
```

LVF skew normal distributions are specified with liberty groups below.

```
ocv_std_dev_cell_rise
ocv_std_dev_cell_fall
ocv_mean_shift_cell_rise
ocv_mean_shift_cell_fall
ocv_skewness_cell_rise
ocv_skewness_cell_fall

ocv_std_dev_rise_transition
ocv_std_dev_fall_transition
ocv_skewness_rise_transition
ocv_skewness_fall_transition
ocv_mean_shift_rise_transition
ocv_mean_shift_fall_transition

ocv_std_dev_rise_constraint
ocv_std_dev_fall_constraint
ocv_skewness_rise_constraint
ocv_skewness_fall_constraint
ocv_mean_shift_rise_constraint
ocv_mean_shift_fall_constraint
```

Power Analysis

OpenSTA also supports static power analysis with the `report_power` command. Probabalistic switching activities are propagated from the input ports to determine switching activities for internal pins.


```

read_liberty sky130hd_tt.lib
read_verilog gcd_sky130hd.v
link_design gcd
read_sdc gcd_sky130hd.sdc
read_spef gcd_sky130hd.spef
set_power_activity -input -activity 0.1
set_power_activity -input_port reset -activity 0
report_power

```

In this example the activity for all inputs is set to 0.1, and then the activity for the reset signal is set to zero because it does not switch during steady state operation.

Group	Internal Power	Switching Power	Leakage Power	Total Power	(Watts)
Sequential	3.27e-04	7.87e-05	2.96e-10	4.06e-04	36.4%
Combinational	2.34e-04	3.10e-04	6.95e-10	5.43e-04	48.7%
Clock	4.68e-05	1.20e-04	2.30e-11	1.67e-04	15.0%
Macro	0.00e+00	0.00e+00	0.00e+00	0.00e+00	0.0%
Pad	0.00e+00	0.00e+00	0.00e+00	0.00e+00	0.0%
Total	6.07e-04	5.09e-04	1.01e-09	1.12e-03	100.0%
	54.4%	45.6%	0.0%		

This example can be found in `examples/power.tcl`.

Gate level simulation results can be used to get a more accurate power estimate. For example, the Icarus verilog simulator can be used to run the test bench `examples/gcd_tb.v` for the gcd design in the previous example.

```

iverilog -o gcd_tb gcd_tb.v
vvp gcd_tb

```

The test bench writes the VCD (Value Change Data) file `gcd_sky130hd.vcd` which can then be read with the `read_vcd` command.

```

read_liberty sky130hd_tt.lib
read_verilog gcd_sky130hd.v
link_design gcd
read_sdc gcd_sky130hd.sdc
read_spef gcd_sky130hd.spef
read_vcd -scope gcd_tb/gcd1 gcd_sky130hd.vcd.gz
report_power

```

This example can be found in `examples/power_vcd.tcl`.

Note that in this simple example design simulation based activities does not significantly change the results.

TCL Interpreter

Keyword arguments to commands may be abbreviated. For example,

```
report_checks -unique
```

is equivalent to the following command.

```
report_checks -unique_paths_to_endpoint
```

The help command lists matching commands and their arguments.

```
> help report*
report_annotated_check [-setup] [-hold] [-recovery] [-removal] [-nochange]
  [-width] [-period] [-max_skew] [-max_lines lines] [-
list_annotated]group_path_count
  [-list_not_annotated] [-constant_arcs]
report_annotated_delay [-cell] [-net] [-from_in_ports] [-to_out_ports]
  [-max_lines lines] [-list_annotated] [-list_not_annotated] [-constant_arcs]
report_arrival pin
report_check_types [-violators] [-verbose] [-scene scene]
  [-format slack_only|end] [-max_delay] [-min_delay] [-recovery] [-removal]
  [-clock_gating_setup] [-clock_gating_hold] [-max_slew] [-min_slew]
  [-max_fanout] [-min_fanout] [-max_capacitance] [-min_capacitance]
  [-min_pulse_width] [-min_period] [-max_skew] [-net net] [-digits digits]
  [-no_line_splits] [> filename] [>> filename]
report_checks [-from from_list|-rise_from from_list|-fall_from from_list]
  [-through through_list|-rise_through through_list|-fall_through
through_list]
  [-to to_list|-rise_to to_list|-fall_to to_list] [-unconstrained]
  [-path_delay min|min_rise|min_fall|max|max_rise|max_fall|min_max]
  [-scene scene] [-group_path_count path_count]
  [-endpoint_path_count path_count]
  [-unique_paths_to_endpoint] [-slack_max slack_max] [-slack_min slack_min]
  [-sort_by_slack] [-path_group group_name]
  [-format full|full_clock|full_clock_expanded|short|end|summary]
...
```

Many reporting commands support redirection of the output to a file much like a Unix shell.

```
report_checks -to out1 > path.log
report_checks -to out2 >> path.log
```

Debugging Timing

Here are some guidelines for debugging your design if static timing does not report any paths, or does not report the expected paths.

Debugging timing problems generally involves using the following commands to follow the propagation of arrival times from a known arrival downstream to understand why the arrival times are not propagating:

```
report_edges
report_arrivals
report_net
```

`report_edges -from` can be used to walk forward and `report_edges -to` to walk backward in the netlist/timing graph. `report_arrivals` shows the min/max rise/fall arrival times with respect to each clock that has a path to the pin. `report_net` shows connections to a net across hierarchy levels.

No paths found

The `report_checks` command only reports paths that are constrained by timing checks or SDC commands such as `set_output_delay`. If the design has only combinational logic (no registers or latches), there are no timing checks, so no paths are reported. Use the `-unconstrained` option to `report_checks` to see unconstrained paths.

```
% report_checks -unconstrained
```

If the design is sequential (has registers or latches) and no paths are reported, it is likely that there is a problem with the clock propagation. Check the timing at a register in the design with the `report_arrivals` command.

```
% report_arrivals r1/CP
(clk ^) r 0.00:0.00 f INF:-INF
(clk v) r INF:-INF f 5.00:5.00
```

In this example the rising edge of the clock "clk" causes the rising arrival min:max time at 0.00, and the falling edge arrives at 5.00. Since the rising edge of the clock causes the rising edge of the register clock pin, the clock path is positive unate.

The clock path should be positive or negative unate. Something is probably wrong with the clock network if it is non-unate. A non-unate clock path will report arrivals similar to the following:

```
% report_arrivals r1/CP
(clk ^) r 0.00:0.00 f 0.00:0.00
(clk v) r 5.00:5.00 f 5.00:5.00
```

Notice that each clock edge causes both rise and fall arrivals at the register clock pin.

If there are no paths to the register clock pin, nothing is printed. Use the `report_edges -to` command to find the gate driving the clock pin.

```
% report_edges -to r1/CP
i1/ZN -> CP wire
^ -> ^ 0.00:0.00
v -> v 0.00:0.00
```

This shows that the gate/pin i1/ZN is driving the clock pin. The `report_edges -to` command can be used to walk backward or forward through the netlist one gate/net at a time. By checking the arrivals with the `report_arrival` command you can determine where the path is broken.

No path reported an endpoint

In order for a timing check to be reported, there must be an arrival time at the data pin (the constrained pin) as well as the timing check clock pin. If `report_checks -to` a register input does not report any paths, check that the input is constrained by a timing check with `report_edges -to`.

```
% report_edges -to r1/D
CP -> D hold
^ -> ^ -0.04:-0.04
^ -> v -0.03:-0.03
CP -> D setup
^ -> ^ 0.09:0.0
^ -> v 0.08:0.08
in1 -> D wire
^ -> ^ 0.00:0.00
v -> v 0.00:0.00
```

This reports the setup and hold checks for the D pin of r1.

Next, check the arrival times at the D and CP pins of the register with `report_arrivals`.

```
% report_arrivals r1/D
(clk1 ^) r 1.00:1.00 f 1.00:1.00
% report_arrivals r1/CP
(clk1 ^) r 0.00:0.00 f INF:-INF
(clk1 v) r INF:-INF f 5.00:5.00
```

If there are no arrivals on an input port of the design, use the `set_input_delay` command to specify the arrival times on the port.

Commands

all_clocks

The `all_clocks` command returns a list of all clocks that have been defined.

all_inputs	<code>[-no_clocks]</code>
-------------------	---------------------------

<code>-no_clocks</code>	Exclude inputs defined as clock sources.
-------------------------	--

The `all_inputs` command returns a list of all input and bidirect ports of the current design.

all_outputs

The `all_outputs` command returns a list of all output and bidirect ports of the design.

all_registers	<code>[-clock <i>clock_names</i></code> <code>[-cells -data_pins -clock_pins -async_pins</code> <code> -output_pins]</code> <code>[-level_sensitive]</code> <code>[-edge_triggered]</code>
----------------------	---

<code>-clock <i>clock_names</i></code>	A list of clock names. Only registers clocked by these clocks are returned.
--	---

<code>-cells</code>	Return a list of register instances.
---------------------	--------------------------------------

<code>-data_pins</code>	Return the register data pins.
-------------------------	--------------------------------

<code>-clock_pins</code>	Return the register clock pins.
--------------------------	---------------------------------

<code>-async_pins</code>	Return the register set/clear pins.
--------------------------	-------------------------------------

<code>-output_pins</code>	Return the register output pins.
---------------------------	----------------------------------

<code>-level_sensitive</code>	Return level-sensitive latches.
-------------------------------	---------------------------------

<code>-edge_triggered</code>	Return edge-triggered registers.
------------------------------	----------------------------------

The `all_registers` command returns a list of register instances or register pins in the design. Options allow the list of registers to be restricted in various ways. The `-clock` keyword restricts the registers to those that are clocked by a set of clocks. The `-cells` option returns the list of registers or latches (the default). The `--data_pins`, `-clock_pins`, `-async_pins` and `-output_pins` options cause `all_registers` to return a list of register pins rather than instances.

check_setup	[-verbose] [-unconstrained_endpoints] [-multiple_clock] [-no_clock] [-no_input_delay] [-loops] [-generated_clocks] [> <i>filename</i>] [>> <i>filename</i>]
-verbose	Show offending objects rather than just error counts.
-unconstrained_endpoints	Check path endpoints for timing constraints (timing check or set_output_delay).
-multiple_clock	Check register/latch clock pins for multiple clocks.
-no_clock	Check register/latch clock pins for a clock.
-no_input_delay	Check for inputs that do not have a set_input_delay command.
-loops	Check for combinational logic loops.
-generated_clocks	Check that generated clock source pins have been defined as clocks.

The `check_setup` command performs sanity checks on the design. Individual checks can be performed with the keywords. If no check keywords are specified all checks are performed. Checks that fail are reported as warnings. If no checks fail nothing is reported. The command returns 1 if there are no warnings for use in scripts.

connect_pin	<i>net</i> <i>port</i> <i>pin</i>
<i>net</i>	A net to add connections to.
<i>port</i>	A port to connect to <i>net</i> .
<i>Pin</i>	A pin to connect to <i>net</i> .

The `connect_pin` command connects a port or instance pin to a net.

create_clock	-period <i>period</i> [-name <i>clock_name</i>] [-waveform <i>edge_list</i>] [-add] [<i>pin_list</i>]
-period <i>period</i>	The clock period.
-name <i>clock_name</i>	The name of the clock.
-waveform <i>edge_list</i>	A list of edge rise and fall time.
-add	Add this clock to the clocks on <i>pin_list</i> .
<i>pin_list</i>	A list of pins driven by the clock.

The `create_clock` command defines the waveform of a clock used by the design.

If no *pin_list* is specified the clock is *virtual*. A virtual clock can be referred to by name in input arrival and departure time commands but is not attached to any pins in the design.

If no clock name is specified the name of the first pin is used as the clock name.

If a waveform is not specified the clock rises at zero and falls at half the clock period. The waveform is a list with time the clock rises as the first element and the time it falls as the second element.

If a clock is already defined on a pin the clock is redefined using the new clock parameters. If multiple clocks drive the same pin, use the `-add` option to prevent the existing definition from being overwritten.

The following command creates a clock with a period of 10 time units that rises at time 0 and falls at 5 time units on the pin named `clk1`.

```
create_clock -period 10 clk1
```

The following command creates a clock with a period of 10 time units that is high at time zero, falls at time 2 and rises at time 8. The clock drives three pins named `clk1`, `clk2`, and `clk3`.

```
create_clock -period 10 -waveform {8 2} -name clk {clk1 clk2 clk3}
```

create_generated_clock	<code>[-name <i>clock_name</i>]</code> <code>-source <i>master_pin</i></code> <code>[-master_clock <i>master_clock</i>]</code> <code>[-divide_by <i>divisor</i>]</code> <code>[-multiply_by <i>multiplier</i>]</code> <code>[-duty_cycle <i>duty_cycle</i>]</code> <code>[-invert]</code> <code>[-edges <i>edge_list</i>]</code> <code>[-edge_shift <i>shift_list</i>]</code> <code>[-add]</code> <i>pin_list</i>
<code>-name <i>clock_name</i></code>	The name of the generated clock.
<code>-source <i>master_pin</i></code>	A pin or port in the fanout of the master clock that is the source of the generated clock.
<code>-master_clock <i>master_clock</i></code>	Use <code>-master_clock</code> to specify which source clock to use when multiple clocks are present on <i>master_pin</i> .
<code>-divide_by <i>divisor</i></code>	Divide the master clock period by <i>divisor</i> .
<code>-multiply_by <i>multiplier</i></code>	Multiply the master clock period by <i>multiplier</i> .
<code>-duty_cycle <i>duty_cycle</i></code>	The percent of the period that the generated clock is high (between 0 and 100).
<code>-invert</code>	Invert the master clock.
<code>-edges <i>edge_list</i></code>	List of master clock edges to use in the generated clock. Edges are numbered from 1. <i>edge_list</i> must be 3 edges long.
<code>-edge_shift <i>shift_list</i></code>	Not supported.
<code>-add</code>	Add this clock to the existing clocks on <i>pin_list</i> .
<i>pin_list</i>	A list of pins driven by the generated clock.

The `create_generated_clock` command is used to generate a clock from an existing clock definition. It is used to model clock generation circuits such as clock dividers and phase locked loops.

The `-divide_by`, `-multiply_by` and `-edges` arguments are mutually exclusive.

The `-multiply_by` option is used to generate a higher frequency clock from the source clock. The period of the generated clock is divided by *multiplier*. The clock *multiplier* must be a positive integer. If a duty cycle is specified the generated clock rises at zero and falls at period * duty_cycle / 100. If no duty cycle is specified the source clock edge times are divided by *multiplier*.

The `-divide_by` option is used to generate a lower frequency clock from the source clock. The clock *divisor* must be a positive integer. If the clock divisor is a power of two the source clock period is multiplied by *divisor*, the clock rise time is the same as the source clock, and the clock fall edge is one half period later. If the clock divisor is not a power of two the source clock waveform edge times are multiplied by *divisor*.

The `-edges` option forms the generated clock waveform by selecting edges from the source clock waveform.

If the `-invert` option is specified the waveform derived above is inverted.

If a clock is already defined on a pin the clock is redefined using the new clock parameters. If multiple clocks drive the same pin, use the `-add` option to prevent the existing definition from being overwritten.

In the example show below generates a clock named `gclk1` on register output pin `r1/Q` by dividing it by four.

```
create_clock -period 10 -waveform {1 8} clk1
create_generated_clock -name gclk1 -source clk1 -divide_by 4 r1/Q
```

The generated clock has a period of 40, rises at time 1 and falls at time 21.

In the example shown below the duty cycle is used to define the derived clock waveform.

```
create_generated_clock -name gclk1 -source clk1 -duty_cycle 50 \
    -multiply_by 2 r1/Q
```

The generated clock has a period of 5, rises at time .5 and falls at time 3.

In the example shown below the first, third and fifth source clock edges are used to define the derived clock waveform.

```
create_generated_clock -name gclk1 -source clk1 -edges {1 3 5} r1/Q
```

The generated clock has a period of 20, rises at time 1 and falls at time 11.

create_voltage_area	<code>[-name name]</code>
	<code>[-coordinate coordinates]</code>
	<code>[-guard_band_x guard_x]</code>
	<code>[-guard_band_y guard_y]</code>
	<code>cells</code>

This command is parsed and ignored by timing analysis.

current_design	[<i>design</i>]
-----------------------	-------------------

current_instance	[<i>instance</i>]
-------------------------	---------------------

<i>instance</i>	Not supported.
-----------------	----------------

define_scene	-mode <i>mode_name</i> -liberty <i>liberty_files</i> -liberty_min <i>liberty_min_files</i> -liberty_max <i>liberty_max_files</i> -spef <i>spef_file</i> -spef_min <i>spef_min_file</i> -spef_max <i>spef_max_file</i>
---------------------	--

<i>mode_name</i>	The SDC mode to use.
------------------	----------------------

<i>liberty_files</i>	List of Liberty files to use.
----------------------	-------------------------------

<i>spef_file</i>	The SPEF parasitics file to use.
------------------	----------------------------------

The `define_scene` command defines a scene for a mode (SDC), liberty files and spef parasitics. Define scenes after reading Liberty libraries and SPEF parasitics.

Liberty files are specified with the name of the liberty library or the filename of the liberty file. If a filename is used, it must be the same as the filename used to read the library with `read_liberty`.

Use `get_scenes` to find defined scenes.

delete_clock	[-all] <i>clocks</i>
---------------------	----------------------

<i>clocks</i>	A list of clocks to remove.
---------------	-----------------------------

delete_from_list	<i>list objects</i>
-------------------------	---------------------

<i>list</i>	A list of objects.
-------------	--------------------

objects A list of objects to delete from list.

delete_generated_clock [-all] *clocks*

clocks A list of generated clocks to remove.

delete_instance *instance*

instance Instance to delete.

The network editing command `delete_instance` removes an instance from the design.

delete_net *net*

net Net to delete.

The network editing command `delete_net` removes a net from the design.

disconnect_pin *net*
 port | *pin* | -all

net The net to disconnect pins from.

port A port to connect to *net*.

pin A pin to connect to *net*.

-all Disconnect all pins from the net.

Disconnects a port or pin from a net. Parasitics connected to the pin are deleted.

elapsed_run_time

Returns the total clock run time in seconds as a float.

find_timing_paths	<pre> [-from <i>from_list</i> -rise_from <i>from_list</i> -fall_from <i>from_list</i>] [-through <i>through_list</i> -rise_through <i>through_list</i> -fall_through <i>through_list</i>] [-to <i>to_list</i> -rise_to <i>to_list</i> -fall_to <i>to_list</i>] [-unconstrained] [-path_delay <i>min</i> <i>min_rise</i> <i>min_fall</i> <i>max</i> <i>max_rise</i> <i>max_fall</i> <i>min_max</i>] [-group_path_count <i>path_count</i>] [-endpoint_path_count <i>endpoint_path_count</i>] [-unique_paths_to_endpoint] [-scene <i>scene</i>] [-slack_max <i>max_slack</i>] [-slack_min <i>min_slack</i>] [-sort_by_slack] [-path_group <i>groups</i>] </pre>
<code>-from <i>from_list</i></code>	Return paths from a list of clocks, instances, ports, register clock pins, or latch data pins.
<code>-rise_from <i>from_list</i></code>	Return paths from the rising edge of clocks, instances, ports, register clock pins, or latch data pins.
<code>-fall_from <i>from_list</i></code>	Return paths from the falling edge of clocks, instances, ports, register clock pins, or latch data pins.
<code>-through <i>through_list</i></code>	Return paths through a list of instances, pins or nets.
<code>-rise_through <i>through_list</i></code>	Return rising paths through a list of instances, pins or nets.
<code>-fall_through <i>through_list</i></code>	Return falling paths through a list of instances, pins or nets.
<code>-to <i>to_list</i></code>	Return paths to a list of clocks, instances, ports or pins.
<code>-rise_to <i>to_list</i></code>	Return rising paths to a list of clocks, instances, ports or pins.
<code>-fall_to <i>to_list</i></code>	Return falling paths to a list of clocks, instances, ports or pins.
<code>-unconstrained</code>	Report unconstrained paths also.

<code>-path_delay min</code>	Return min path (hold) checks.
<code>-path_delay min_rise</code>	Return min path (hold) checks for rising endpoints.
<code>-path_delay min_fall</code>	Return min path (hold) checks for falling endpoints.
<code>-path_delay max</code>	Return max path (setup) checks.
<code>-path_delay max_rise</code>	Return max path (setup) checks for rising endpoints.
<code>-path_delay max_fall</code>	Return max path (setup) checks for falling endpoints.
<code>-path_delay min_max</code>	Return max and max path (setup and hold) checks.
<code>-group_path_count</code> <i>path_count</i>	The number of paths to return in each path group.
<code>-endpoint_path_count</code> <i>endpoint_path_count</i>	The number of paths to return for each endpoint.
<code>-unique_paths_to_endpoint</code>	Return multiple paths to an endpoint that traverse different pins without showing multiple paths with different rise/fall transitions.
<code>-scene scene</code>	Return paths for one process corner.
<code>-slack_max max_slack</code>	Return paths with slack less than <i>max_slack</i> .
<code>-slack_min min_slack</code>	Return paths with slack greater than <i>min_slack</i> .
<code>-sort_by_slack</code>	Sort paths by slack rather than slack within path groups.
<code>-path_group groups</code>	Return paths in path groups. Paths in all groups are returned if this option is not specified.

The `find_timing_paths` command returns a list of path objects for scripting. Use the `get_property` function to access properties of the paths.

get_cells	[-hierarchical] [-hsc <i>separator</i>] [-filter <i>expr</i>] [-regexp] [-nocase] [-quiet] [-of_objects <i>objects</i>] [<i>patterns</i>]
-hierarchical	Searches hierarchy levels below the current instance for matches.
-hsc <i>separator</i>	Character to use to separate hierarchical instance names in <i>patterns</i> .
-filter <i>expr</i>	A filter expression of the form " <i>property==value</i> " where <i>property</i> is a property supported by the get_property command. See the section "Filter Expressions" for additional forms.
-regexp	Use regular expression matching instead of glob pattern matching.
-nocase	Ignore case when matching. Only valid with -regexp.
-quiet	Do not warn if no matches are found.
-of_objects <i>objects</i>	The name of a pin or net, a list of pins returned by get_pins, or a list of nets returned by get_nets. The -hierarchical option cannot be used with -of_objects.
<i>patterns</i>	A list of instance name patterns.

The get_cells command returns a list of all cell instances that match *patterns*.

get_clocks	[-regexp] [-nocase] [-filter <i>expr</i>] [-quiet] <i>patterns</i>
-regexp	Use regular expression matching instead of glob pattern matching.
-nocase	Ignore case when matching. Only valid with -regexp.

<code>-filter <i>expr</i></code>	A filter expression of the form <code>"property==value"</code> where <i>property</i> is a property supported by the <code>get_property</code> command. See the section "Filter Expressions" for additional forms.
<code>-quiet</code>	Do not warn if no matches are found.
<code><i>patterns</i></code>	A list of clock name patterns.

The `get_clocks` command returns a list of all clocks that have been defined.

<code>get_fanin</code>	<code>-to <i>sink_list</i></code> <code>[-flat]</code> <code>[-only_cells]</code> <code>[-startpoints_only]</code> <code>[-levels <i>level_count</i>]</code> <code>[-pin_levels <i>pin_count</i>]</code> <code>[-trace_arcs timing enabled all]</code>
<code>-to <i>sink_list</i></code>	List of pins, ports, or nets to find the fanin of. For nets, the fanin of driver pins on the nets are returned.
<code>-flat</code>	With <code>-flat</code> pins in the fanin at any hierarchy level are returned. Without <code>-flat</code> only pins at the same hierarchy level as the sinks are returned.
<code>-only_cells</code>	Return the instances connected to the pins in the fanin.
<code>-startpoints_only</code>	Only return pins that are startpoints.
<code>-level <i>level_count</i></code>	Only return pins within <i>level_count</i> instance traversals.
<code>-pin_levels <i>pin_count</i></code>	Only return pins within <i>pin_count</i> pin traversals.
<code>-trace_arcs timing</code>	Only trace through timing arcs that are not disabled.
<code>-trace_arcs enabled</code>	Only trace through timing arcs that are not disabled.
<code>-trace_arcs all</code>	Trace through all arcs, including disabled ones.

The `get_fanin` command returns traverses the design from *sink_list* pins, ports or nets backwards and return the fanin pins or instances.

get_fanout	-from <i>source_list</i> [-flat] [-only_cells] [-endpoints_only] [-levels <i>level_count</i>] [-pin_levels <i>pin_count</i>] [-trace_arcs timing enabled all]
-from <i>source_list</i>	List of pins, ports, or nets to find the fanout of. For nets, the fanout of load pins on the nets are returned.
-flat	With -flat pins in the fanin at any hierarchy level are returned. Without -flat only pins at the same hierarchy level as the sinks are returned.
-only_cells	Return the instances connected to the pins in the fanout.
-endpoints_only	Only return pins that are endpoints.
-level <i>level_count</i>	Only return pins within <i>level_count</i> instance traversals.
-pin_levels <i>pin_count</i>	Only return pins within <i>pin_count</i> pin traversals.
-trace_arcs timing	Only trace through timing arcs that are not disabled.
-trace_arcs enabled	Only trace through timing arcs that are not disabled.
-trace_arcs all	Trace through all arcs, including disabled ones.

The `get_fanout` command returns traverses the design from *source_list* pins, ports or nets backwards and return the fanout pins or instances.

get_full_name	<i>object</i>
<i>object</i>	A library, cell, port, instance, pin or timing arc object.

Return the name of *object*. Equivalent to `[get_property object full_name]`.

get_lib_cells	[-of_objects <i>objects</i>] [-hsc <i>separator</i>] [-filter <i>expr</i>] [-regexp] [-nocase] [-quiet] <i>patterns</i>
----------------------	--

-of_objects <i>objects</i>	A list of instance objects.
-hsc <i>separator</i>	Character that separates the library name and cell name in <i>patterns</i> . Defaults to '/'.
-filter <i>expr</i>	A filter expression of the form " <i>property==value</i> " where <i>property</i> is a property supported by the get_property command. See the section "Filter Expressions" for additional forms.
-regexp	Use regular expression matching instead of glob pattern matching.
-nocase	Ignore case when matching. Only valid with -regexp.
-quiet	Do not warn if no matches are found.
<i>patterns</i>	A list of library cell name patterns of the form library_name/cell_name.

The get_lib_cells command returns a list of library cells that match *pattern*. The library name can be prepended to the cell name pattern with the *separator* character, which defaults to hierarchy_separator.

get_lib_pins	[-of_objects <i>objects</i>] [-hsc <i>separator</i>] [-filter <i>expr</i>] [-regexp] [-nocase] [-quiet] <i>patterns</i>
---------------------	--

-of_objects <i>objects</i>	A list of library cell objects.
-hsc <i>separator</i>	Character that separates the library name, cell name and port name in <i>pattern</i> . Defaults to '/'.

<code>-filter <i>expr</i></code>	A filter expression of the form " <i>property==value</i> " where <i>property</i> is a property supported by the <code>get_property</code> command. See the section "Filter Expressions" for additional forms.
<code>-regexp</code>	Use regular expression matching instead of glob pattern matching.
<code>-nocase</code>	Ignore case when matching. Only valid with <code>-regexp</code> .
<code>-quiet</code>	Do not warn if no matches are found.
<code><i>patterns</i></code>	A list of library port name patterns of the form library_name/cell_name/port_name.

The `get_lib_pins` command returns a list of library ports that match *pattern*. Use *separator* to separate the library and cell name patterns from the port name in *pattern*.

<code>get_libs</code>	<code>[-filter <i>expr</i>]</code> <code>[-regexp]</code> <code>[-nocase]</code> <code>[-quiet]</code> <code><i>patterns</i></code>
<code>-filter <i>expr</i></code>	A filter expression of the form " <i>property==value</i> " where <i>property</i> is a property supported by the <code>get_property</code> command. See the section "Filter Expressions" for additional forms.
<code>-regexp</code>	Use regular expression matching instead of glob pattern matching.
<code>-nocase</code>	Ignore case when matching. Only valid with <code>-regexp</code> .
<code>-quiet</code>	Do not warn if no matches are found.
<code><i>patterns</i></code>	A list of library name patterns.

The `get_libs` command returns a list of clocks that match *patterns*.

get_nets	[-hierarchical] [-hsc <i>separator</i>] [-filter <i>expr</i>] [-regexp] [-nocase] [-quiet] [-of_objects <i>objects</i>] [<i>patterns</i>]
-hierarchical	Searches hierarchy levels below the current instance for matches.
-hsc <i>separator</i>	Character that separates the library name, cell name and port name in <i>pattern</i> . Defaults to '/'. <i>pattern</i> .
-filter <i>expr</i>	A filter expression of the form " <i>property==value</i> " where <i>property</i> is a property supported by the get_property command. See the section "Filter Expressions" for additional forms.
-regexp	Use regular expression matching instead of glob pattern matching.
-nocase	Ignore case when matching. Only valid with -regexp.
-quiet	Do not warn if no matches are found.
-of_objects <i>objects</i>	The name of a pin or instance, a list of pins returned by get_pins, or a list of instances returned by get_cells. The -hierarchical option cannot be used with -of_objects.
<i>patterns</i>	A list of net name patterns.

The get_nets command returns a list of all nets that match *patterns*.

get_name	<i>object</i>
<i>object</i>	A library, cell, port, instance, pin or timing arc object.

Return the name of *object*. Equivalent to [get_property *object* name].

get_pins	[-hierarchical] [-hsc <i>separator</i>] [-filter <i>expr</i>] [-regexp] [-nocase] [-quiet] [-of_objects <i>objects</i>] [<i>patterns</i>]
-hierarchical	Searches hierarchy levels below the current instance for matches.
-hsc <i>separator</i>	Character that separates the library name, cell name and port name in <i>pattern</i> . Defaults to '/'.
-filter <i>expr</i>	A filter expression of the form " <i>property==value</i> " where <i>property</i> is a property supported by the get_property command. See the section "Filter Expressions" for additional forms.
-nocase	Ignore case when matching. Only valid with -regexp.
-quiet	Do not warn if no matches are found.
-of_objects <i>objects</i>	The name of a net or instance, a list of nets returned by get_nets, or a list of instances returned by get_cells. The -hierarchical option cannot be used with -of_objects.
<i>patterns</i>	A list of pin name patterns.

The get_pins command returns a list of all instance pins that match *patterns*.

A useful idiom to find the driver pin for a net is the following.

```
get_pins -of_objects [get_net net_name] -filter "direction==output"
```

get_ports	[-filter <i>expr</i>] [-regexp] [-nocase] [-quiet] [-of_objects <i>objects</i>] [<i>patterns</i>]
------------------	--

<code>-filter <i>expr</i></code>	A filter expression of the form " <i>property==value</i> " where <i>property</i> is a property supported by the <code>get_property</code> command. See the section "Filter Expressions" for additional forms.
<code>-regexp</code>	Use regular expression matching instead of glob pattern matching.
<code>-nocase</code>	Ignore case when matching. Only valid with <code>-regexp</code> .
<code>-quiet</code>	Do not warn if no matches are found.
<code>-of_objects <i>objects</i></code>	The name of net or a list of nets returned by <code>get_nets</code> .
<i>patterns</i>	A list of port name patterns.

The `get_ports` command returns a list of all top level ports that match *patterns*.

<code>get_property</code>	<code>[-object_type <i>object_type</i>]</code> <i>object</i> <i>property</i>
<code>-object_type <i>object_type</i></code>	The type of <i>object</i> when it is specified as a name. cell pin net port clock library library_cell library_pin timing_arc
<i>object</i>	An object returned by <code>get_cells</code> , <code>get_pins</code> , <code>get_nets</code> , <code>get_ports</code> , <code>get_clocks</code> , <code>get_libs</code> , <code>get_lib_cells</code> , <code>get_lib_pins</code> , or <code>get_timing_arcs</code> , or object name. <code>-object_type</code> is required if <i>object</i> is a name.
<i>property</i>	A property name.

The properties for different objects types are shown below.

cell (SDC lib_cell)

base_name
filename
full_name
library
name

clock

full_name
is_generated
is_propagated
is_virtual
name
period
sources

edge

delay_max_fall
delay_min_fall
delay_max_rise
delay_min_rise
full_name
from_pin
sense
to_pin

instance (SDC cell)

cell
full_name
is_buffer
is_clock_gate
is_hierarchical
is_inverter
is_macro
is_memory
liberty_cell
name
ref_name

liberty_cell (SDC lib_cell)

area
base_name
dont_use
filename
full_name
is_buffer
is_inverter
is_memory
library
name

liberty_port (SDC lib_pin)

capacitance
direction
drive_resistance
drive_resistance_max_fall
drive_resistance_max_rise
drive_resistance_min_fall
drive_resistance_min_rise
full_name
intrinsic_delay
intrinsic_delay_max_fall
intrinsic_delay_max_rise
intrinsic_delay_min_fall
intrinsic_delay_min_rise
is_register_clock
lib_cell
name

library

filename (Liberty library only)
name
full_name

net

full_name
name

path (PathEnd)

endpoint
endpoint_clock
endpoint_clock_pin
slack
startpoint
startpoint_clock
points

pin

activity (activity in transitions per second, duty cycle, origin)
origin is one of
global set_power_activity -global
input set_power_activity -input
user set_power_activity -input_ports -pins
vcd read_vcd
saif read_saif
propagated propagated from upstream activities
clock SDC create_clock or create_generated_clock
constant constant pins propagated from verilog tie high/low,
 set_case_analysis, set_logic_one/zero/dc

slew_max_fall
slew_max_rise
slew_min_fall
slew_min_rise
clocks
clock_domains
direction
full_name
is_hierarchical
is_port
is_register_clock
lib_pin_name
name
slack_max
slack_max_fall
slack_max_rise
slack_min
slack_min_fall
slack_min_rise

port

activity
slew_max_fall
slew_max_rise
slew_min_fall
slew_min_rise
direction
full_name
liberty_port
name
slack_max
slack_max_fall
slack_max_rise
slack_min
slack_min_fall
slack_min_rise

point (PathRef)

arrival
pin
required
slack

get_scenes	<code>[-mode <i>mode_name</i>]</code> <code><i>scene_name</i></code>
<i>mode_name</i>	Get the scenes for <i>mode_name</i> .
<i>scene_name</i>	A scene name pattern.

The `get_scenes` command is used to find the scenes matching a pattern or that use an SDC mode.

get_timing_edges	<code>[-from <i>from_pins</i>]</code> <code>[-to <i>to_pins</i>]</code> <code>[-of_objects <i>objects</i>]</code> <code>[-filter <i>expr</i>]</code> <code>[<i>patterns</i>]</code>
<code>-from <i>from_pin</i></code>	A list of pins.
<code>-to <i>to_pin</i></code>	A list of pins.
<code>-of_objects <i>objects</i></code>	A list of instances or library cells. The <code>-from</code> and <code>-to</code> options cannot be used with <code>-of_objects</code> .
<code>-filter <i>expr</i></code>	A filter expression of the form <code>"property==value"</code> where <i>property</i> is a property supported by the <code>get_property</code> command. See the section "Filter Expressions" for additional forms.

The `get_timing_edges` command returns a list of timing edges (arcs) to, from or between pins. The result can be passed to `get_property` or `set_disable_timing`.

group_path	-name <i>group_name</i> [-weight <i>weight</i>] [-critical_range <i>range</i>] [-from <i>from_list</i> -rise_from <i>from_list</i> -fall_from <i>from_list</i>] [-through <i>through_list</i> -rise_through <i>through_list</i> -fall_through <i>through_list</i>] [-to <i>to_list</i> -rise_to <i>to_list</i> -fall_to <i>to_list</i>] [-default]
-name <i>group_name</i>	The name of the path group.
-weight <i>weight</i>	Not supported.
-critical_range <i>range</i>	Not supported.
-from <i>from_list</i>	Group paths from a list of clocks, instances, ports, register clock pins, or latch data pins.
-rise_from <i>from_list</i>	Group paths from the rising edge of clocks, instances, ports, register clock pins, or latch data pins.
-fall_from <i>from_list</i>	Group paths from the falling edge of clocks, instances, ports, register clock pins, or latch data pins.
-through <i>through_list</i>	Group paths through a list of instances, pins or nets.
-rise_through <i>through_list</i>	Group rising paths through a list of instances, pins or nets.
-fall_through <i>through_list</i>	Group falling paths through a list of instances, pins or nets.
-to <i>to_list</i>	Group paths to a list of clocks, instances, ports or pins.
-rise_to <i>to_list</i>	Group rising paths to a list of clocks, instances, ports or pins.
-fall_to <i>to_list</i>	Group falling paths to a list of clocks, instances, port-s or pins.
-default	Restore the paths in the path group -from/-to/-through/-to to their default path group.

The `group_path` command is used to group paths reported by the `report_checks` command. See `set_false_path` for a description of allowed *from_list*, *through_list* and *to_list* objects.

include	<code>[-echo -e]</code> <code>[-verbose -v]</code> <i>filename</i> <code>[> log_filename]</code> <code>[>> log_filename]</code>
<code>-echo -e</code>	Print each command before evaluating it.
<code>-verbose -v</code>	Print each command before evaluating it as well as the result it returns.
<i>filename</i>	The name of the file containing commands to read.
<code>> log_filename</code>	Redirect command output to log_filename.
<code>>> log_filename</code>	Redirect command output and append log_filename.

Read STA/SDC/Tcl commands from *filename*.

The `include` command stops and reports any errors encountered while reading a file unless `sta_continue_on_error` is 1.

link_design	<code>[-no_black_boxes]</code> <i>cell_name</i>
<code>-no_black_boxes</code>	Do not make empty "black box" cells for instances that reference undefined cells.
<i>cell_name</i>	The top level module/cell name of the design hierarchy to link.

Link (elaborate, flatten) the the top level cell *cell_name*. The design must be linked after reading netlist and library files. The default value of *cell_name* is the current design.

The linker creates empty "block box" cells for instances the reference undefined cells when the variable `link_create_black_boxes` is true. When `link_create_black_boxes` is false an error is reported and the link fails.

The `link_design` command returns 1 if the link succeeds and 0 if it fails.

make_instance	<i>inst_path</i> <i>lib_cell</i>
----------------------	-------------------------------------

<i>inst_path</i>	A hierarchical instance name.
<i>lib_cell</i>	The library cell of the new instance.

The `make_instance` command makes an instance of library cell *lib_cell*.

make_net	<i>net_name_list</i>
-----------------	----------------------

<i>net_name_list</i>	A list of net names.
----------------------	----------------------

Creates a net for each hierarchical net name.

read_liberty	[-corner <i>corner</i>] [-min] [-max] [-infer_latches] <i>filename</i>
---------------------	---

-corner <i>corner</i>	Use the library for process corner <i>corner</i> delay calculation.
-----------------------	---

-min	Use library for min delay calculation.
------	--

-max	Use library for max delay calculation.
------	--

<i>filename</i>	The liberty file name to read.
-----------------	--------------------------------

The `read_liberty` command reads a Liberty format library file. The first library that is read sets the units used by SDC/TCL commands and reporting. The `include_file` attribute is supported.

Some Liberty libraries do not include `latch` groups for cells that are describe transparent latches. In that situation the `-infer_latches` command flag can be used to infer the latches. The timing arcs required for a latch to be inferred should look like the following:

```

cell (inferred_latch) {
  pin(D) {
    direction : input ;
    timing () {
      related_pin : "E" ;
      timing_type : setup_falling ;
    }
    timing () {
      related_pin : "E" ;
      timing_type : hold_falling ;
    }
  }
  pin(E) {
    direction : input;
  }
  pin(Q) {
    direction : output ;
    timing () {
      related_pin : "D" ;
    }
    timing () {
      related_pin : "E" ;
      timing_type : rising_edge ;
    }
  }
}

```

In this example a positive level-sensitive latch is inferred.

Files compressed with gzip are automatically uncompressed.

read_saif	<code>[-scope <i>scope</i>]</code> <code><i>filename</i></code>
<i>scope</i>	The SAIF scope of the current design to extract simulation data. Typically the test bench name and design under test instance name. Scope levels are separated with <code>`/'</code> .
<i>filename</i>	The name of the SAIF file to read.

The `read_saif` command reads a SAIF (Switching Activity Interchange Format) file from a Verilog simulation and extracts pin activities and duty cycles for use in power estimation. Files compressed with gzip are supported. Annotated activities are propagated to the fanout of the annotated pins.

read_sdc	[-mode <i>mode_name</i>] [-echo] <i>filename</i>
<i>mode_name</i>	Mode for the SDC commands in the file.
-echo	Print each command before evaluating it.
<i>filename</i>	SDC command file.

Read SDC commands from *filename*.

If the mode does not exist it is created. Multiple SDC files can append commands to a mode by using the -mode_name argument for each one. If no -mode argument is used the commands are added to the current mode.

The read_sdc command stops and reports any errors encountered while reading a file unless sta_continue_on_error is 1.

Files compressed with gzip are automatically uncompressed.

read_sdf	[-scene <i>scene</i>] [-unescaped_dividers] <i>filename</i>
<i>scene</i>	Scene delays to annotate.
-unescaped_dividers	With this option path names in the SDF do not have to escape hierarchy dividers when the path name is escaped. For example, the escaped Verilog name "\inst1/inst2 " can be referenced as "inst1/inst2". The correct SDF name is "inst1\inst2", since the divider does not represent a change in hierarchy in this case.
<i>filename</i>	The name of the SDF file to read.

Read SDF delays from a file. The min and max values in the SDF tuples are used to annotate the delays for *corner*. The typical values in the SDF tuples are ignored. If multiple scenes are defined -scene must be specified. SDC annotation for mcm analysis must follow the scene definitions.

Files compressed with gzip are automatically uncompressed.

INCREMENT is supported as an alias for INCREMENTAL.

The following SDF statements are not supported.

PORT
INSTANCE wildcards

read_spef	<code>[-name <i>name</i>] [-keep_capacitive_coupling] [-coupling_reduction_factor <i>factor</i>] [-reduce] [-path <i>path</i>] <i>filename</i></code>
<i>name</i>	The name of the SPEF parasitics to use for defining scenes. The default is the base name of <i>filename</i> .
<i>path</i>	Hierarchical block instance path to annotate with parasitics.
<code>-keep_capacitive_coupling</code>	Keep coupling capacitors in parasitic networks rather than converting them to grounded capacitors.
<code>-coupling_reduction_factor</code> <i>factor</i>	Factor to multiply coupling capacitance by when reducing parasitic networks. The default value is 1.0.
<i>filename</i>	The name of the parasitics file to read.

The `read_spef` command reads a file of net parasitics in SPEF format. Use the `-report_parasitic_annotation` command to check for nets that are not annotated.

Files compressed with gzip are automatically uncompressed.

Separate min/max parasitics can be annotated for each scene mode/corner.

```
read_spef -name min spef1
read_spef -name max spef2
define_scene -mode model1 -spef_min min -spef_max max
```

Coupling capacitors are multiplied by the `-coupling_reduction_factor` when a parasitic network is reduced.

The following SPEF constructs are ignored.


```

*DESIGN_FLOW (all values are ignored)
*S slews
*D driving cell
*I pin capacitances (library cell capacitances are used instead)
*Q r_net load poles
*K r_net load residues

```

If the SPEF file contains triplet values the first value is used.

Parasitic networks (DSPEF) can be annotated on hierarchical blocks using the -path argument to specify the instance path to the block. Parasitic networks in the higher level netlist are stitched together at the hierarchical pins of the blocks.

read_vcd	[-scope <i>scope</i>] [-mode <i>mode_name</i>] <i>filename</i>
<i>scope</i>	The VCD scope of the current design to extract simulation data. Typically the test bench name and design under test instance name. Scope levels are separated with `/'.
<i>mode_name</i>	Mode to annotate activities.
<i>filename</i>	The name of the VCD file to read.

The `read_vcd` command reads a VCD (Value Change Dump) file from a Verilog simulation and extracts pin activities and duty cycles for use in power estimation. Files compressed with gzip are supported. Annotated activities are propagated to the fanout of the annotated pins.

read_verilog	<i>filename</i>
<i>filename</i>	The name of the verilog file to read.

The `read_verilog` command reads a gate level verilog netlist. After all verilog netlist and Liberty libraries are read the design must be linked with the `link_design` command.

Verilog 2001 module port declaratations are supported. An example is shown below.

```

module top (input in1, in2, clk1, clk2, clk3,
            output out);

```

Files compressed with gzip are automatically uncompressed.

replace_cell	<i>instance_list</i> <i>replacement_cell</i>
---------------------	---

<i>instance_list</i>	A list of instances to swap the cell.
----------------------	---------------------------------------

<i>replacement_cell</i>	The replacement lib cell.
-------------------------	---------------------------

The `replace_cell` command changes the cell of an instance. The replacement cell must have the same port list (number, name, and order) as the instance's existing cell for the replacement to be successful.

replace_activity_annotation	<code>[-report_unannotated]</code> <code>[-report_annotated]</code>
------------------------------------	--

<code>-report_unannotated</code>	Report unannotated pins.
----------------------------------	--------------------------

<code>-report_annotated</code>	Report annotated pins.
--------------------------------	------------------------

Report a summary of pins that are annotated by `read_vcd`, `read_saif` or `set_power_activity`. Sequential internal pins and hierarchical pins are ignored.

report_annotated_check	<code>[-setup]</code> <code>[-hold]</code> <code>[-recovery]</code> <code>[-removal]</code> <code>[-nochange]</code> <code>[-width]</code> <code>[-period]</code> <code>[-max_skew]</code> <code>[-max_line lines]</code> <code>[-report_annotated]</code> <code>[-report_unannotated]</code> <code>[-constant_arcs]</code>
-------------------------------	--

<code>-setup</code>	Report annotated setup checks.
---------------------	--------------------------------

<code>-hold</code>	Report annotated hold checks.
--------------------	-------------------------------

<code>-recovery</code>	Report annotated recovery checks.
------------------------	-----------------------------------

<code>-removal</code>	Report annotated removal checks.
-----------------------	----------------------------------

<code>-nochange</code>	Report annotated nochange checks.
------------------------	-----------------------------------

<code>-width</code>	Report annotated width checks.
---------------------	--------------------------------

<code>-period</code>	Report annotated period checks.
<code>-max_skew</code>	Report annotated max skew checks.
<code>-max_line lines</code>	Maximum number of lines listed by the <code>-report_annotated</code> and <code>-report_unannotated</code> options.
<code>-report_annotated</code>	Report annotated timing arcs.
<code>-report_unannotated</code>	Report unannotated timing arcs.
<code>-constant_arcs</code>	Report separate annotation counts for arcs disabled by logic constants (<code>set_logic_one</code> , <code>set_logic_zero</code>).

The `report_annotated_check` command reports a summary of SDF timing check annotation. The `-report_annotated` and `-report_unannotated` options can be used to list arcs that are annotated or not annotated.

<code>report_annotated_delay</code>	<code>[-cell]</code> <code>[-net]</code> <code>[-from_in_ports]</code> <code>[-to_out_ports]</code> <code>[-max_lines lines]</code> <code>[-report_annotated]</code> <code>[-report_unannotated]</code> <code>[-constant_arcs]</code>
<code>-cell</code>	Report annotated cell delays.
<code>-net</code>	Report annotated internal net delays.
<code>-from_in_ports</code>	Report annotated delays from input ports.
<code>-to_out_ports</code>	Report annotated delays to output ports.
<code>-max_lines lines</code>	Maximum number of lines listed by the <code>-report_annotated</code> and <code>-report_unannotated</code> options.
<code>-report_annotated</code>	Report annotated timing arcs.
<code>-report_unannotated</code>	Report unannotated timing arcs.
<code>-constant_arcs</code>	Report separate annotation counts for arcs disabled by logic constants (<code>set_logic_one</code> , <code>set_logic_zero</code>).

The `report_annotated_delay` command reports a summary of SDF delay annotation. Without the `-from_in_ports` and `-to_out_ports` options arcs to and from top level ports are not reported. The `-report_annotated` and `-report_unannotated` options can be used to list arcs that are annotated or not annotated.

report_checks	<pre> [-from <i>from_list</i> -rise_from <i>from_list</i> -fall_from <i>from_list</i>] [-through <i>through_list</i> -rise_through <i>through_list</i> -fall_through <i>through_list</i>] [-to <i>to_list</i> -rise_to <i>to_list</i> -fall_to <i>to_list</i>] [-unconstrained] [-path_delay min min_rise min_fall max max_rise max_fall min_max] [-group_path_count <i>path_count</i>] [-endpoint_path_count <i>endpoint_path_count</i>] [-unique_paths_to_endpoint] [-unique_edges_to_endpoint] [-scenes <i>scenes</i>] [-slack_max <i>max_slack</i>] [-slack_min <i>min_slack</i>] [-sort_by_slack] [-path_group <i>groups</i>] [-format end full short summary full_clock full_clock_expanded json] [-fields <i>fields</i>] [-digits <i>digits</i>] [-no_line_split] [> <i>filename</i>] [>> <i>filename</i>] </pre>
<code>-from <i>from_list</i></code>	Report paths from a list of clocks, instances, ports, register clock pins, or latch data pins.
<code>-rise_from <i>from_list</i></code>	Report paths from the rising edge of clocks, instances, ports, register clock pins, or latch data pins.
<code>-fall_from <i>from_list</i></code>	Report paths from the falling edge of clocks, instances, ports, register clock pins, or latch data pins.

<code>-through through_list</code>	Report paths through a list of instances, pins or nets.
<code>-rise_through through_list</code>	Report rising paths through a list of instances, pins or nets.
<code>-fall_through through_list</code>	Report falling paths through a list of instances, pins or nets.
<code>-to to_list</code>	Report paths to a list of clocks, instances, ports or pins.
<code>-rise_to to_list</code>	Report rising paths to a list of clocks, instances, ports or pins.
<code>-fall_to to_list</code>	Report falling paths to a list of clocks, instances, ports or pins.
<code>-unconstrained</code>	Report unconstrained paths also. The unconstrained path group is not reported without this option.
<code>-path_delay min</code>	Report min path (hold) checks.
<code>-path_delay min_rise</code>	Report min path (hold) checks for rising endpoints.
<code>-path_delay min_fall</code>	Report min path (hold) checks for falling endpoints.
<code>-path_delay max</code>	Report max path (setup) checks.
<code>-path_delay max_rise</code>	Report max path (setup) checks for rising endpoints.
<code>-path_delay max_fall</code>	Report max path (setup) checks for falling endpoints.
<code>-path_delay min_max</code>	Report max and max path (setup and hold) checks.
<code>-group_path_count path_count</code>	The number of paths to report in each path group. The default is 1.
<code>-endpoint_path_count endpoint_path_count</code>	The number of paths to report for each endpoint. The default is 1.
<code>-unique_paths_to_endpoint</code>	When multiple paths to an endpoint are specified with <code>-endpoint_path_count</code> , many of the paths may differ only in the rise/fall edges of the pins in the paths. With this option only the worst path through the set of pins is reported.

<code>-unique_edges_to_endpoint</code>	When multiple paths to an endpoint are specified with <code>-endpoint_path_count</code> , conditional timing arcs result in paths that through the same pins and rise/fall edges. With this option only the worst path through the set of pins and rise/fall edges is reported.
<code>scenes</code>	Report paths for one process corner. The default is to report paths for all process corners.
<code>max_slack</code>	Only report paths with less slack than <code>max_slack</code> .
<code>min_slack</code>	Only report paths with more slack than <code>min_slack</code> .
<code>-sort_by_slack</code>	Sort paths by slack rather than slack grouped by path group.
<code>groups</code>	List of path groups to report. The default is to report all path groups.
<code>-format end</code>	Report path ends in one line with delay, required time and slack.
<code>-format full</code>	Report path start and end points and the path. This is the default path type.
<code>-format full_clock</code>	Report path start and end points, the path, and the source and and target clock paths.
<code>-format full_clock_expanded</code>	Report path start and end points, the path, and the source and and target clock paths. If the clock is generated and propagated, the path from the clock source pin is also reported.
<code>-format short</code>	Report only path start and end points.
<code>-format summary</code>	Report only path ends with delay.
<code>-format json</code>	Report in json format. <code>-fields</code> is ignored.
<code>fields</code>	List of <code>capacitance slew input_pins hierarchical_pins net fanout src_attr variation</code>
<code>digits</code>	The number of digits after the decimal point to report. The default value is the variable <code>sta_report_default_digits</code> .
<code>-no_line_splits</code>	Do not split long lines into multiple lines.

The `report_checks` command reports paths in the design. Paths are reported in groups by capture clock, unlocked path delays, gated clocks and unconstrained.

See `set_false_path` for a description of allowed `from_list`, `through_list` and `to_list` objects.

report_check_types	[-scenes <i>scenes</i>] [-violators] [-verbose] [-fields <i>fields</i>] [-format slack_only end] [-max_delay] [-min_delay] [-recovery] [-removal] [-clock_gating_setup] [-clock_gating_hold] [-max_slew] [-min_slew] [-min_pulse_width] [-min_period] [-digits <i>digits</i>] [-no_split_lines] [> <i>filename</i>] [>> <i>filename</i>]
<i>scenes</i>	Report checks for some scenes. The default value is all scenes.
-violators	Report all violated timing and design rule constraints.
-verbose	Use a verbose output format.
-format slack_only	Report the minimum slack for each timing check.
-format end	Report the endpoint for each check.
<i>fields</i>	List of capacitance slew input_pins hierarchical_pins net fanout src_attr variation
-max_delay	Report setup and max delay path delay constraints.
-min_delay	Report hold and min delay path delay constraints.
-recovery	Report asynchronous recovery checks.
-removal	Report asynchronous removal checks.
-clock_gating_setup	Report gated clock enable setup checks.
-clock_gating_hold	Report gated clock hold setup checks.

<code>-max_slew</code>	Report max transition design rule checks.
<code>-max_skew</code>	Report max skew design rule checks.
<code>-min_pulse_width</code>	Report min pulse width design rule checks.
<code>-min_period</code>	Report min period design rule checks.
<code>-min_slew</code>	Report min slew design rule checks.
<code>-digits <i>digits</i></code>	The number of digits after the decimal point to report. The default value is the variable <code>sta_report_default_digits</code> .
<code>-no_split_lines</code>	Do not split long lines into multiple lines.

The `report_check_types` command reports the slack for each type of timing and design rule constraint. The keyword options allow a subset of the constraint types to be reported.

<code>report_clock_latency</code>	<code>[-clocks <i>clocks</i>]</code> <code>[-scenes <i>scenes</i>]</code> <code>[-include_internal_latency]</code> <code>[-digits <i>digits</i>]</code>
<i>clocks</i>	The clocks to report. The default value is all c
<i>scenes</i>	Report clocks for <i>scenes</i> . The default value is all clocks in <i>scenes</i> modes.
<code>-include_internal_latency</code>	Include internal clock latency from liberty min/max_clock_tree_path timing groups.
<i>digits</i>	The number of digits to report for delays.

Report the clock network latency.

<code>report_clock_min_period</code>	<code>[-clocks <i>clocks</i>]</code> <code>[-scenes <i>scenes</i>]</code> <code>[-include_port_paths]</code>
<i>clocks</i>	The clocks to report.
<code>-include_port_paths</code>	Include paths from input port and to output ports.

Report the minimum period and maximum frequency for *clocks*. If the `-clocks` argument is not specified all clocks are reported. The minimum period is determined by examining the smallest slack paths between

registers the rising edges of the clock or between falling edges of the clock. Paths between different clocks, different clock edges of the same clock, level sensitive latches, or paths constrained by `set_multicycle_path`, `set_max_path` are not considered.

report_clock_properties [*clock_names*]

clock_names List of clock names to report.

The `report_clock_properties` command reports the period and rise/fall edge times for each clock that has been defined.

report_clock_skew [-setup|-hold]
 [-clocks *clocks*]
 [-scenes *scenes*]
 [-include_internal_latency]
 [-digits *digits*]

-setup Report skew for setup checks.

-hold Report skew for hold checks.

clocks The clocks to report. The default value is all clocks in *scenes* modes.

scenes Report clocks for *scenes*. The default value is all scenes.

-include_internal_latency Include internal clock latency from liberty min/max_clock_tree_path timing groups.

-digits *digits* The number of digits to report for delays.

Report the maximum difference in clock arrival between every source and target register that has a path between the source and target registers.

report_dcalc [-from *from_pin*]
 [-to *to_pin*]
 [-scene *scene*]
 [-min]
 [-max]
 [-digits *digits*]
 [> *filename*]
 [>> *filename*]

from_pin Report delay calculations for timing arcs from instance input pin *from_pin*.

<i>to_pin</i>	Report delay calculations for timing arcs to instance output pin <i>to_pin</i> .
<i>scene</i>	Report paths for process <i>scene</i> . The <code>-scene</code> keyword is required if more than one process corner is defined.
<code>-min</code>	Report delay calculation for min delays.
<code>-max</code>	Report delay calculation for max delays.
<code>-digits <i>digits</i></code>	The number of digits after the decimal point to report. The default is <code>sta_report_default_digits</code> .

The `report_dcalc` command shows how the delays between instance pins are calculated. It is useful for debugging problems with delay calculation.

report_disabled_edges

The `report_disabled_edges` command reports disabled timing arcs along with the reason they are disabled. Each disabled timing arc is reported as the instance name along with the from and to ports of the arc. The disable reason is shown next. Arcs that are disabled with `set_disable_timing` are reported with `constraint` as the reason. Arcs that are disabled by constants are reported with `constant` as the reason along with the constant instance pin and value. Arcs that are disabled to break combinational feedback loops are reported with `loop` as the reason.

```
> report_disabled_edges
u1 A B constant B=0
```

report_edges	<code>[-from <i>from_pin</i>]</code> <code>[-to <i>to_pin</i>]</code> <code>[-report_variation]</code> <code>[-digits <i>digits</i>]</code>
<i>from_pin</i>	Report edges/timing arcs from pin <i>from_pin</i> .
<i>to_pin</i>	Report edges/timing arcs to pin <i>to_pin</i> .
<code>-report_variation</code>	
<i>digits</i>	The number of digits after the decimal point to report. The default value is the variable <code>sta_report_default_digits</code> .

Report the edges/timing arcs and their delays in the timing graph from/to/between pins.

report_instance	<i>instance_path</i> [> <i>filename</i>] [>> <i>filename</i>]
------------------------	---

<i>instance_path</i>	Hierarchical path to an instance.
----------------------	-----------------------------------

report_lib_cell	<i>cell_name</i> [> <i>filename</i>] [>> <i>filename</i>]
------------------------	---

<i>cell_name</i>	The name of a library cell.
------------------	-----------------------------

Describe the liberty library cell *cell_name*.

report_net	[-digits <i>digits</i>] <i>net_path</i> [> <i>filename</i>] [>> <i>filename</i>]
-------------------	--

<i>digits</i>	The number of digits after the decimal point to report. The default value is the variable <code>sta_report_default_digits</code> .
---------------	--

<i>net_path</i>	Hierarchical path to a net.
-----------------	-----------------------------

Report the connections and capacitance of a net.

report_parasitic_annotation	[-report_unannotated] [> <i>filename</i>] [>> <i>filename</i>]
------------------------------------	--

-report_unannotated	Report unannotated and partially annotated nets.
---------------------	--

Report SPEF parasitic annotation completeness.

report_power	[-instances <i>instances</i>] [-highest_power_instances <i>count</i>] [-digits <i>digits</i>] [> <i>filename</i>] [>> <i>filename</i>]
---------------------	---

`-instances instances` Report the power for each instance of *instances*. If the instance is hierarchical the total power for the instances inside the hierarchical instance is reported.

`-highest_power_instances` Report the power for the *count* highest power instances.

`count`

`-digits digits` The number of digits after the decimal point to report. The default value is the variable `sta_report_default_digits`.

The `report_power` command uses static power analysis based on propagated or annotated pin activities in the circuit using Liberty power models. The internal, switching, leakage and total power are reported. Design power is reported separately for combinational, sequential, macro and pad groups. Power values are reported in watts.

The `read_vcd` or `read_saif` commands can be used to read activities from a file based on simulation. If no simulation activities are available, the `set_power_activity` command should be used to set the activity of input ports or pins in the design. The default input activity and duty for inputs are 0.1 and 0.5 respectively. The activities are propagated from annotated input ports or pins through gates and used in the power calculations.

Group	Internal Power	Switching Power	Leakage Power	Total Power	
Sequential	3.29e-06	3.41e-08	2.37e-07	3.56e-06	92.4%
Combinational	1.86e-07	3.31e-08	7.51e-08	2.94e-07	7.6%
Macro	0.00e+00	0.00e+00	0.00e+00	0.00e+00	0.0%
Pad	0.00e+00	0.00e+00	0.00e+00	0.00e+00	0.0%
Total	3.48e-06	6.72e-08	3.12e-07	3.86e-06	100.0%
	90.2%	1.7%	8.1%		

report_slews `[-scenes scenes]`
`[-report_variation]`
`[-digits digits]`
pin

scenes Report slews for process for scenes process corners.

`-report_variation` Report SSTA distribution parameters.

`-digits digits` The number of digits after the decimal point to report. The default value is the variable `sta_report_default_digits`.

pin

Report the slews at *pin*

report_tns	[-min] [-max] [-digits <i>digits</i>]
-max	Report the total max/setup slack.
-min	Report the total min/hold slack.
-digits <i>digits</i>	The number of digits after the decimal point to report. The default value is the variable <code>sta_report_default_digits</code> .

Report the total negative slack.

report_units

Report the units used for command arguments and reporting.

```
report_units
time 1ns
capacitance 1pF
resistance 1kohm
voltage 1v
current 1A
power 1pW
distance 1um
```

report_wns	[-min] [-max] [-digits <i>digits</i>]
-max	Report the worst max/setup slack.
-min	Report the worst min/hold slack.
-digits <i>digits</i>	The number of digits after the decimal point to report. The default value is the variable <code>sta_report_default_digits</code> .

Report the worst negative slack. If the worst slack is positive, zero is reported.

report_worst_slack	[-min] [-max] [-digits <i>digits</i>]
---------------------------	--

<code>-max</code>	Report the worst max/setup slack.
<code>-min</code>	Report the worst min/hold slack.
<code>-digits <i>digits</i></code>	The number of digits after the decimal point to report. The default value is the variable <code>sta_report_default_digits</code> .

<code>set_assigned_check</code>	<code>-setup -hold -recovery -removal</code> <code>[-rise]</code> <code>[-fall]</code> <code>[-scene <i>scene</i>]</code> <code>[-min]</code> <code>[-max]</code> <code>[-from <i>from_pins</i>]</code> <code>[-to <i>to_pins</i>]</code> <code>[-clock <i>rise</i> <i>fall</i>]</code> <code>[-cond <i>sdf_cond</i>]</code> <code>[-worst]</code> <code><i>margin</i></code>
<code>-setup</code>	Annotate setup timing checks.
<code>-hold</code>	Annotate hold timing checks.
<code>-recovery</code>	Annotate recovery timing checks.
<code>-removal</code>	Annotate removal timing checks.
<code>-rise</code>	Annotate rising delays.
<code>-fall</code>	Annotate falling delays.
<code>scene</code>	The name of a scene. The <code>-scene</code> keyword is required if more than one scene is defined.
<code>-min</code>	Annotate the minimum value of the process corner.
<code>-max</code>	Annotate the maximum value of the process corner.
<code><i>from_pins</i></code>	A list of pins for the clock.

<i>to_pins</i>	A list of pins for the data.
<code>-clock rise fall</code>	The timing check clock pin transition.
<i>margin</i>	The timing check margin.

The `set_assigned_check` command is used to annotate the timing checks between two pins on an instance. The annotated delay overrides the calculated delay. This command is an interactive way to back-annotate delays like an SDF file.

set_assigned_delay	<code>-cell -net</code> [<code>-rise</code>] [<code>-fall</code>] [<code>-scene scene</code>] [<code>-min</code>] [<code>-max</code>] [<code>-from from_pins</code>] [<code>-to to_pins</code>] <i>delay</i>
<code>-cell</code>	Annotate the delays between two pins on an instance.
<code>-net</code>	Annotate the delays between two pins on a net.
<code>-rise</code>	Annotate the rising delays.
<code>-fall</code>	Annotate the falling delays.
<i>scene</i>	The name of a scene. The <code>-scene</code> keyword is required if more than one scene is defined.
<code>-min</code>	Annotate the minimum delays.
<code>-max</code>	Annotate the maximum delays.
<i>from_pins</i>	A list of pins.
<i>to_pins</i>	A list of pins.
<i>delay</i>	The delay between <i>from_pins</i> and <i>to_pins</i> .

The `set_assigned_delay` command is used to annotate the delays between two pins on an instance or net. The annotated delay overrides the calculated delay. This command is an interactive way to back-annotate delays like an SDF file.

set_assigned_transition [-rise]
[-fall]
[-scene *scene*]
[-min]
[-max]
slew
pin_list

-rise Annotate the rising transition.

-fall Annotate the falling transition.

scene Annotate delays for scene.

-min Annotate the minimum transition time.

-max Annotate the maximum transition time.

slew The pin transition time.

pin_list A list of pins.

The `set_assigned_transition` command is used to annotate the transition time (*slew*) of a pin. The annotated transition time overrides the calculated transition time.

set_case_analysis 0|1|zero|one|rise|rising|fall|falling
port_or_pin_list

port_or_pin_list A list of ports or pins.

The `set_case_analysis` command sets the signal on a port or pin to a constant logic value. No paths are propagated from constant pins. Constant values set with the `set_case_analysis` command are propagated through downstream gates.

Conditional timing arcs with mode groups are controlled by logic values on the instance pins.

set_clock_gating_check [-setup *setup_time*]
[-hold *hold_time*]
[-rise]
[-fall]
[-high]
[-low]
[*objects*]

<code>-setup <i>setup_time</i></code>	Clock enable setup margin.
<code>-hold <i>hold_time</i></code>	Clock enable hold margin.
<code>-rise</code>	The setup/hold margin is for the rising edge of the clock enable.
<code>-fall</code>	The setup/hold margin is for the falling edge of the clock enable.
<code>-high</code>	The gating clock is active high (pin and instance objects only).
<code>-low</code>	The gating clock is active low (pin and instance objects only).
<code><i>objects</i></code>	A list of clocks, instances, pins or ports.

The `set_clock_gating_check` command is used to add setup or hold timing checks for data signals used to gate clocks.

If no objects are specified the setup/hold margin is global and applies to all clock gating circuits in the design. If neither of the `-rise` and `-fall` options are used the setup/hold margin applies to the rising and falling edges of the clock gating signal.

Normally the library cell function is used to determine the active state of the clock. The clock is active high for AND/NAND functions and active low for OR/NOR functions. The `-high` and `-low` options are used to specify the active state of the clock for other cells, such as a MUX.

If multiple `set_clock_gating_check` commands apply to a clock gating instance the priority of the commands is shown below (highest to lowest priority).

```

clock enable pin
instance
clock pin
clock
global

```

set_clock_groups	<code>[-name <i>name</i>]</code> <code>[-logically_exclusive]</code> <code>[-physically_exclusive]</code> <code>[-asynchronous]</code> <code>[-allow_paths]</code> <code>-group <i>clocks</i></code>
-------------------------	---

<code>-name <i>name</i></code>	The clock group name.
--------------------------------	-----------------------

<code>-logically_exclusive</code>	The clocks in different groups do not interact logically but can be physically present on the same chip. Paths between clock groups are considered for noise analysis.
<code>-physically_exclusive</code>	The clocks in different groups cannot be present at the same time on a chip. Paths between clock groups are not considered for noise analysis.
<code>-asynchronous</code>	The clock groups are asynchronous. Paths between clock groups are considered for noise analysis.
<code>-allow_paths</code>	
<i>clocks</i>	A list of clocks in the group.

The `set_clock_groups` command is used to define groups of clocks that interact with each other. Clocks in different groups do not interact and paths between them are not reported. Use a `-group` argument for each clock group.

<code>set_clock_latency</code>	<code>[-source]</code> <code>[-clock <i>clock</i>]</code> <code>[-rise]</code> <code>[-fall]</code> <code>[-min]</code> <code>[-max]</code> <i>delay</i> <i>objects</i>
<code>-source</code>	The latency is at the clock source.
<code>-clock <i>clock</i></code>	If multiple clocks are defined at a pin this use this option to specify the latency for a specific clock.
<code>-rise</code>	The latency is for the rising edge of the clock.
<code>-fall</code>	The latency is for the falling edge of the clock.
<code>-min</code>	<i>delay</i> is the minimum latency.
<code>-max</code>	<i>delay</i> is the maximum latency.
<i>delay</i>	Clock source or insertion delay.
<i>objects</i>	A list of clocks, pins or ports.

The `set_clock_latency` command describes expected delays of the clock tree when analyzing a design using ideal clocks. Use the `-source` option to specify latency at the clock source, also known as insertion delay. Source latency is delay in the clock tree that is external to the design or a clock tree internal to an instance that implements a complex logic function.

`set_clock_latency` removes propagated clock properties for the clocks and pins objects.

set_clock_transition	[-rise] [-fall] [-min] [-max] <i>transition</i> <i>clocks</i>
-rise	Set the transition time for the rising edge of the clock.
-fall	Set the transition time for the falling edge of the clock.
-min	Set the min transition time.
-max	Set the min transition time.
<i>transition</i>	Clock transition time (slew).
<i>clocks</i>	A list of clocks.

The `set_clock_transition` command describes expected transition times of the clock tree when analyzing a design using ideal clocks.

set_clock_uncertainty	[-from -rise_from -fall_from <i>from_clock</i>] [-to -rise_to -fall_to <i>to_clock</i>] [-rise] [-fall] [-setup] [-hold] <i>uncertainty</i> <i>[objects]</i>
-from <i>from_clock</i>	Inter-clock uncertainty source clock.
-to <i>to_clock</i>	Inter-clock uncertainty target clock.
-rise	Inter-clock target clock rise edge, alternative to -rise_to. Inter-clock target clock rise edge, alternative to -rise_to.

<code>-fall</code>	Inter-clock target clock rise edge, alternative to <code>-fall_to</code> .
<code>-setup</code>	<i>uncertainty</i> is for setup checks.
<code>-hold</code>	<i>uncertainty</i> is for hold checks.
<i>uncertainty</i>	Clock uncertainty.
<i>objects</i>	A list of clocks, ports or pins.

The `set_clock_uncertainty` command specifies the uncertainty or jitter in a clock. The uncertainty for a clock can be specified on its source pin or port, or the clock itself.

```
set_clock_uncertainty .1 [get_clock clk1]
```

Inter-clock uncertainty between the source and target clocks of timing checks is specified with the `-from` | `-rise_from` | `-fall_from` and `-to` | `-rise_to` | `-fall_to` arguments .

```
set_clock_uncertainty -from [get_clock clk1] -to [get_clocks clk2] .1
```

The following commands are equivalent.

```
set_clock_uncertainty -from [get_clock clk1] -rise_to [get_clocks clk2] .1
set_clock_uncertainty -from [get_clock clk1] -to [get_clocks clk2] -rise .1
```

set_cmd_units	<code>[-capacitance <i>cap_unit</i>]</code> <code>[-resistance <i>res_unit</i>]</code> <code>[-time <i>time_unit</i>]</code> <code>[-voltage <i>voltage_unit</i>]</code> <code>[-current <i>current_unit</i>]</code> <code>[-power <i>power_unit</i>]</code> <code>[-distance <i>distance_unit</i>]</code>
<code>-capacitance <i>cap_unit</i></code>	The capacitance scale factor followed by 'f' .
<code>-resistance <i>res_unit</i></code>	The resistance scale factor followed by 'ohm' .
<code>-time <i>time_unit</i></code>	The time scale factor followed by 's'.
<code>-voltage <i>voltage_unit</i></code>	The voltage scale factor followed by 'v' .
<code>-current <i>current_unit</i></code>	The current scale factor followed by 'A' .
<code>-power <i>power_unit</i></code>	The power scale factor followed by 'w' .

`-distance distance_unit` The distance scale factor followed by 'm'.

The `set_cmd_units` command is used to change the units used by the STA command interpreter when parsing commands and reporting results. The default units are the units specified in the first Liberty library file that is read.

Units are specified as a scale factor followed by a unit name. The scale factors are as follows.

```
M 1E+6
k 1E+3
m 1E-3
u 1E-6
n 1E-9
p 1E-12
f 1E-15
```

An example of the `set_units` command is shown below.

```
set_cmd_units -time ns -capacitance pF -current mA -voltage V
              -resistance kOhm -distance um
```

set_data_check	<code>[-from -rise_from -fall_from <i>from_pin</i>]</code> <code>[-to -rise_to -fall_to <i>to_pin</i>]</code> <code>[-setup]</code> <code>[-hold]</code> <code>[-clock <i>clock</i>]</code> <i>margin</i>
-----------------------	--

<code>-from <i>from_pin</i></code>	A pin used as the timing check reference.
------------------------------------	---

<code>-to <i>to_pin</i></code>	A pin that the setup/hold check is applied to.
--------------------------------	--

<code>-setup</code>	Add a setup timing check.
---------------------	---------------------------

<code>-hold</code>	Add a hold timing check.
--------------------	--------------------------

<code>-clock <i>clock</i></code>	The setup/hold check clock.
----------------------------------	-----------------------------

<i>margin</i>	The setup or hold time margin.
---------------	--------------------------------

The `set_data_check` command is used to add a setup or hold timing check between two pins.

set_disable_inferred_clock_gating *objects*

objects A list of clock gating instances, clock gating pins, or clock enable pins.

The `set_disable_inferred_clock_gating` command disables clock gating checks on a clock gating instance, clock gating pin, or clock gating enable pin.

set_disable_timing [-from *from_port*]
 [-to *to_port*]
 objects

-from *from_port*

-to *to_port*

objects A list of instances, ports, pins, cells, cell/port, or library/cell/port.

The `set_disable_timing` command is used to disable paths through pins in the design. There are many different forms of the command depending on the objects specified in *objects*.

All timing paths through an instance are disabled when *objects* contains an instance. Timing checks in the instance are *not* disabled.

```
set_disable_timing u2
```

The `-from` and `-to` options can be used to restrict the disabled path to those from, to or between specific pins on the instance.

```
set_disable_timing -from A u2
set_disable_timing -to Z u2
set_disable_timing -from A -to Z u2
```

A list of top level ports or instance pins can also be disabled.

```
set_disable_timing u2/Z
set_disable_timing in1
```

Timing paths through all instances of a library cell in the design can be disabled by naming the cell using a hierarchy separator between the library and cell name. Paths from or to a cell port can be disabled with the `-from` and `-to` options or a port name after library and cell names.

```

set_disable_timing liberty1/snl_bufx2
set_disable_timing -from A liberty1/snl_bufx
set_disable_timing -to Z liberty1/snl_bufx
set_disable_timing liberty1/snl_bufx2/A

```

set_drive	[-rise] [-fall] [-max] [-min] <i>resistance</i> <i>ports</i>
-rise	Set the drive rise resistance.
-fall	Set the drive fall resistance.
-max	Set the maximum resistance.
-min	Set the minimum resistance.
<i>resistance</i>	The external drive resistance.
<i>ports</i>	A list of ports.

The `set_drive` command describes the resistance of an input port external driver.

set_driving_cell	[-lib_cell <i>cell_name</i>] [-library <i>library</i>] [-rise] [-fall] [-min] [-max] [-pin <i>pin</i>] [-from_pin <i>from_pin</i>] [-input_transition_rise <i>trans_rise</i>] [-input_transition_fall <i>trans_fall</i>] <i>ports</i>
-lib_cell <i>cell_name</i>	The driving cell.
-library <i>library</i>	The driving cell library.
-rise	Set the driving cell for a rising edge.

<code>-fall</code>	Set the driving cell for a falling edge.
<code>-max</code>	Set the driving cell for max delays.
<code>-min</code>	Set the driving cell for min delays.
<code>-pin <i>pin</i></code>	The output port of the driving cell.
<code>-from_pin <i>from_pin</i></code>	Use timing arcs from <i>from_pin</i> to the output pin.
<code>-input_transition_rise <i>trans_rise</i></code>	The transition time for a rising input at <i>from_pin</i> .
<code>-input_transition_fall <i>trans_fall</i></code>	The transition time for a falling input at <i>from_pin</i> .
<code><i>ports</i></code>	A list of ports.

The `set_driving_cell` command describes an input port external driver.

<code>set_false_path</code>	<code>[-setup]</code> <code>[-hold]</code> <code>[-rise]</code> <code>[-fall]</code> <code>[-from <i>from_list</i>]</code> <code>[-rise_from <i>from_list</i>]</code> <code>[-fall_from <i>from_list</i>]</code> <code>[-through <i>through_list</i>]</code> <code>[-rise_through <i>through_list</i>]</code> <code>[-fall_through <i>through_list</i>]</code> <code>[-to <i>to_list</i>]</code> <code>[-rise_to <i>to_list</i>]</code> <code>[-fall_to <i>to_list</i>]</code> <code>[-reset_path]</code>
<code>-setup</code>	Apply to setup checks.
<code>-hold</code>	Apply to hold checks.
<code>-rise</code>	Apply to rising path edges.
<code>-fall</code>	Apply to falling path edges.

<code>-reset_path</code>	Remove any matching <code>set_false_path</code> , <code>set_multicycle_path</code> , <code>set_max_delay</code> , <code>set_min_delay</code> exceptions first.
<code>-from from_list</code>	A list of clocks, instances, ports or pins.
<code>-through through_list</code>	A list of instances, pins or nets.
<code>-to to_list</code>	A list of clocks, instances, ports or pins.

The `set_false_path` command disables timing along a path from, through and to a group of design objects.

Objects in `from_list` can be clocks, register/latch instances, or register/latch clock pins. The `-rise_from` and `-fall_from` keywords restrict the false paths to a specific clock edge.

Objects in `through_list` can be nets, instances, instance pins, or hierarchical pins,. The `-rise_through` and `-fall_through` keywords restrict the false paths to a specific path edge that traverses through the object.

Objects in `to_list` can be clocks, register/latch instances, or register/latch clock pins. The `-rise_to` and `-fall_to` keywords restrict the false paths to a specific transition at the path end.

<code>set_fanout_load</code>	<i>fanout</i> <i>port_list</i>
-------------------------------------	-----------------------------------

This command is ignored.

<code>set_hierarchy_separator</code>	<i>separator</i>
---	------------------

<i>separator</i>	Character used to separate hierarchical names.
------------------	--

Set the character used to separate names in a hierarchical instance, net or pin name. This separator is used by the command interpreter to read arguments and print results. The default separator is `'/'`.

<code>set_ideal_latency</code>	<code>[-rise] [-fall] [-min] [-max] delay objects</code>
---------------------------------------	--

The `set_ideal_latency` command is parsed but ignored.

<code>set_ideal_network</code>	<code>[-no_propagation] objects</code>
---------------------------------------	--

The `set_ideal_network` command is parsed but ignored.

set_ideal_transition	<code>[-rise] [-fall] [-min] [-max] <i>transition_time objects</i></code>
-----------------------------	---

The `set_ideal_transition` command is parsed but ignored.

set_input_delay	<code>[-rise] [-fall] [-max] [-min] [-clock <i>clock</i>] [-clock_fall] [-reference_pin <i>ref_pin</i>] [-source_latency_included] [-network_latency_included] [-add_delay] <i>delay</i> <i>port_pin_list</i></code>
<code>-rise</code>	Set the arrival time for the rising edge of the input.
<code>-fall</code>	Set the arrival time for the falling edge of the input.
<code>-max</code>	Set the maximum arrival time.
<code>-min</code>	Set the minimum arrival time.
<code>-clock <i>clock</i></code>	The arrival time is from <i>clock</i> .
<code>-clock_fall</code>	The arrival time is from the falling edge of <i>clock</i> .
<code>-reference_pin <i>ref_pin</i></code>	The arrival time is with respect to the clock that arrives at <i>ref_pin</i> .
<code>-source_latency_included</code>	Do not add the clock source latency (insertion delay) to the delay value.
<code>-network_latency_included</code>	Do not add the clock latency to the delay value when the clock is ideal.
<code>-add_delay</code>	Add this arrival to any existing arrivals.
<i>delay</i>	The arrival time after <i>clock</i> .
<i>pin_port_list</i>	A list of pins or ports.

The `set_input_delay` command is used to specify the arrival time of an input signal.

The following command sets the min, max, rise and fall times on the `in1` input port 1.0 time units after the rising edge of `clk1`.

```
set_input_delay -clock clk1 1.0 [get_ports in1]
```

Use multiple commands with the `-add_delay` option to specify separate arrival times for min, max, rise and fall times or multiple clocks. For example, the following specifies separate arrival times with respect to clocks `clk1` and `clk2`.

```
set_input_delay -clock clk1 1.0 [get_ports in1]
set_input_delay -add_delay -clock clk2 2.0 [get_ports in1]
```

The `-reference_pin` option is used to specify an arrival time with respect to the arrival on a pin in the clock network. For propagated clocks, the input arrival time is relative to the clock arrival time at the reference pin (the clock source latency and network latency from the clock source to the reference pin). For ideal clocks, input arrival time is relative to the reference pin clock source latency. With the `-clock_fall` flag the arrival time is relative to the falling transition at the reference pin. If no clocks arrive at the reference pin the `set_input_delay` command is ignored. If no `-clock` is specified the arrival time is with respect to all clocks that arrive at the reference pin. The `-source_latency_included` and `-network_latency_included` options cannot be used with `-reference_pin`.

Paths from inputs that do not have an arrival time defined by `set_input_delay` are not reported. Set the `sta_input_port_default_clock` variable to 1 to report paths from inputs without a `set_input_delay`.

set_input_transition	<code>[-rise]</code> <code>[-fall]</code> <code>[-max]</code> <code>[-min]</code> <i>transition</i> <i>port_list</i>
<code>-rise</code>	Set the rising edge transition.
<code>-fall</code>	Set the falling edge transition.
<code>-max</code>	Set the minimum transition time.
<code>-min</code>	Set the maximum transition time.
<i>transition</i>	The transition time (slew).
<i>port_list</i>	A list of ports.

The `set_input_transition` command is used to specify the transition time (slew) of an input signal.

set_level_shifter_strategy [-rule *rule_type*]

This command is parsed and ignored by timing analysis.

set_level_shifter_threshold [-voltage *voltage*]

This command is parsed and ignored by timing analysis.

set_load [-rise]
 [-fall]
 [-max]
 [-min]
 [-subtract_pin_load]
 [-pin_load]
 [-wire_load]
 capacitance
 objects

-rise Set the external port rising capacitance (ports only).

-fall Set the external port falling capacitance (ports only).

-max Set the max capacitance.

-min Set the min capacitance.

-subtract_pin_load Subtract the capacitance of all instance pins connected to the net from *capacitance* (nets only). If the resulting capacitance is negative, zero is used. Pin capacitances are ignored by delay calculation when this option is used.

-pin_load *capacitance* is external instance pin capacitance (ports only).

-wire_load *capacitance* is external wire capacitance (ports only).

capacitance The capacitance, in library capacitance units.

objects A list of nets or ports.

The `set_load` command annotates wire capacitance on a net or external capacitance on a port. There are four different uses for the `set_load` command:

```

set_load -wire_load port external port wire capacitance
set_load -pin_load port external port pin capacitance
set_load port same as -pin_load
set_load net net wire capacitance

```

External port capacitance can be annotated separately with the `-pin_load` and `-wire_load` options. Without the `-pin_load` and `-wire_load` options pin capacitance is annotated.

When annotating net wire capacitance with the `-subtract_pin_load` option the capacitance of all instance pins connected to the net is subtracted from *capacitance*. Setting the capacitance on a net overrides SPEF parasitics for delay calculation.

```

set_logic_dc port_list

```

```

port_pin_list List of ports or pins.

```

Set a port or pin to a constant unknown logic value. No paths are propagated from constant pins.

```

set_logic_one port_list

```

```

port_pin_list List of ports or pins.

```

Set a port or pin to a constant logic one value. No paths are propagated from constant pins. Constant values set with the `set_logic_one` command are **not** propagated through downstream gates.

```

set_logic_zero port_list

```

```

port_pin_list List of ports or pins.

```

Set a port or pin to a constant logic zero value. No paths are propagated from constant pins. Constant values set with the `set_logic_zero` command are **not** propagated through downstream gates.

```

set_max_area area

```

```

area

```

The `set_max_area` command is ignored during timing but is included in SDC files that are written.

```

set_max_capacitance capacitance
                      objects

```

capacitance

objects List of ports or cells.

The `set_max_capacitance` command is ignored during timing but is included in SDC files that are written.

set_max_delay	<code>[-rise]</code> <code>[-fall]</code> <code>[-from from_list]</code> <code>[-rise_from from_list]</code> <code>[-fall_from from_list]</code> <code>[-through through_list]</code> <code>[-rise_through through_list]</code> <code>[-fall_through through_list]</code> <code>[-to to_list]</code> <code>[-rise_to to_list]</code> <code>[-fall_to to_list]</code> <code>[-ignore_clock_latency]</code> <code>[-probe]</code> <code>[-reset_path]</code> <i>delay</i>
<code>-rise</code>	Set max delay for rising paths.
<code>-fall</code>	Set max delay for falling paths.
<code>-from from_list</code>	A list of clocks, instances, ports or pins.
<code>-through through_list</code>	A list of instances, pins or nets.
<code>-to to_list</code>	A list of clocks, instances, ports or pins.
<code>-ignore_clock_latency</code>	Ignore clock latency at the source and target registers.
<code>-probe</code>	Do not break paths at internal pins (non startpoints).
<code>-reset_path</code>	Remove any matching <code>set_false_path</code> , <code>set_multicycle_path</code> , <code>set_max_delay</code> , <code>set_min_delay</code> exceptions first.
<i>delay</i>	The maximum delay.

The `set_max_delay` command constrains the maximum delay through combinational logic paths. See `set_false_path` for a description of allowed *from_list*, *through_list* and *to_list* objects. If the *to_list* ends at a timing check the setup/hold time is included in the path delay.

When the `-ignore_clock_latency` option is used clock latency at the source and destination of the path delay is ignored. The constraint is reported in the default path group (****default****) rather than the clock path group when the path ends at a timing check.

set_max_dynamic_power *power* [*unit*]

The `set_max_dynamic_power` command is ignored during timing but is included in SDC files that are written.

set_max_fanout	<i>fanout</i>
	<i>objects</i>

fanout

<i>objects</i>	List of ports or cells.
----------------	-------------------------

The `set_max_fanout` command is ignored during timing but is included in SDC files that are written.

set_max_leakage_power	<i>power</i> [<i>unit</i>]
------------------------------	------------------------------

The `set_max_leakage_power` command is ignored during timing but is included in SDC files that are written.

set_max_time_borrow	<i>delay</i>
	<i>objects</i>

<i>delay</i>	The maximum time the latches can borrow.
--------------	--

<i>objects</i>	List of clocks, instances or pins.
----------------	------------------------------------

The `set_max_time_borrow` command specifies the maximum amount of time that latches can borrow. Time borrowing is the time that a data input to a transparent latch arrives after the latch opens.

```
set_max_transition      [-data_path]
                        [-clock_path]
                        [-rise]
                        [-fall]
                        transition
                        objects
```

`-data_path` Set the max slew for data paths.

<code>-clock_path</code>	Set the max slew for clock paths.
--------------------------	-----------------------------------

<code>-rise</code>	Set the max slew for rising paths.
<code>-fall</code>	Set the max slew for falling paths.
<code>transition</code>	The maximum slew/transition time.
<code>objects</code>	List of clocks, ports or designs.

The `set_max_transition` command specifies the maximum transition time (slew) design rule checked by the `report_check_types -max_transition` command.

If specified for a design, the default maximum transition is set for the design.

If specified for a clock, the maximum transition is applied to all pins in the clock domain. The `-clock_path` option restricts the maximum transition to clocks in clock paths. The `-data_path` option restricts the maximum transition to clocks data paths. The `-clock_path`, `-data_path`, `-rise` and `-fall` options only apply to clock objects.

<code>set_min_capacitance</code>	<i>capacitance</i> <i>objects</i>
<i>capacitance</i>	Minimum capacitance.
<i>objects</i>	List of ports or cells.

The `set_min_capacitance` command is ignored during timing but is included in SDC files that are written.

<code>set_min_delay</code>	<code>[-rise]</code> <code>[-fall]</code> <code>[-from from_list]</code> <code>[-rise_from from_list]</code> <code>[-fall_from from_list]</code> <code>[-through through_list]</code> <code>[-rise_through through_list]</code> <code>[-fall_through through_list]</code> <code>[-to to_list]</code> <code>[-rise_to to_list]</code> <code>[-fall_to to_list]</code> <code>[-ignore_clock_latency]</code> <code>[-probe]</code> <code>[-reset_path]</code> <i>delay</i>
-----------------------------------	---

<code>-rise</code>	Set min delay for rising paths.
<code>-fall</code>	Set min delay for falling paths.
<code>-from <i>from_list</i></code>	A list of clocks, instances, ports or pins.
<code>-through <i>through_list</i></code>	A list of instances, pins or nets.
<code>-to <i>to_list</i></code>	A list of clocks, instances, ports or pins.
<code>-ignore_clock_latency</code>	Ignore clock latency at the source and target registers.
<code>-probe</code>	Do not break paths at internal pins (non startpoints).
<code>-reset_path</code>	Remove any matching <code>set_false_path</code> , <code>set_multicycle_path</code> , <code>set_max_delay</code> , <code>set_min_delay</code> exceptions first.
<code><i>delay</i></code>	The minimum delay.

The `set_min_delay` command constrains the minimum delay through combinational logic. See `set_false_path` for a description of allowed *from_list*, *through_list* and *to_list* objects. If the *to_list* ends at a timing check the setup/hold time is included in the path delay.

When the `-ignore_clock_latency` option is used clock latency at the source and destination of the path delay is ignored. The constraint is reported in the default path group (**default**) rather than the clock path group when the path ends at a timing check.

<code>set_min_pulse_width</code>	<code>[-high]</code> <code>[-low]</code> <i>min_width</i> <i>objects</i>
<code>-high</code>	Set the minimum high pulse width.
<code>-low</code>	Set the minimum low pulse width.
<i>min_width</i>	
<i>objects</i>	List of pins, instances or clocks.

If `-low` and `-high` are not specified the minimum width applies to both high and low pulses.

<code>set_mode</code>	<i>mode_name</i>
------------------------------	------------------

Set the mode for SDC commands in the TCL interpreter. If mode *mode_name* does not exist, it is created. When modes are created the default mode is deleted.

set_multicycle_path	[-setup] [-hold] [-rise] [-fall] [-start] [-end] [-from <i>from_list</i>] [-rise_from <i>from_list</i>] [-fall_from <i>from_list</i>] [-through <i>through_list</i>] [-rise_through <i>through_list</i>] [-fall_through <i>through_list</i>] [-to <i>to_list</i>] [-rise_to <i>to_list</i>] [-fall_to <i>to_list</i>] [-reset_path] <i>path_multiplier</i>
-setup	Set cycle count for setup checks.
-hold	Set cycle count for hold checks.
-rise	Set cycle count for rising path edges.
-fall	Set cycle count for falling path edges.
-start	Multiply the source clock period by <i>period_multiplier</i> .
-end	Multiply the target clock period by <i>period_multiplier</i> .
-from <i>from_list</i>	A list of clocks, instances, ports or pins.
-through <i>through_list</i>	A list of instances, pins or nets.
-to <i>to_list</i>	A list of clocks, instances, ports or pins.
-reset_path	Remove any matching <code>set_false_path</code> , <code>set_multicycle_path</code> , <code>set_max_delay</code> , <code>set_min_delay</code> exceptions first.
<i>path_multiplier</i>	The number of clock periods to add to the path required time.

Normally the path between two registers or latches is assumed to take one clock cycle. The `set_multicycle_path` command overrides this assumption and allows multiple clock cycles for a timing check. See `set_false_path` for a description of allowed *from_list*, *through_list* and *to_list* objects.

set_operating_conditions	<code>[-analysis_type single bc_wc on_chip_variation]</code> <code>[-library lib]</code> <code>[condition]</code> <code>[-min min_condition]</code> <code>[-max max_condition]</code> <code>[-min_library min_lib]</code> <code>[-max_library max_lib]</code>
<code>-analysis_type single</code>	Use one operating condition for min and max paths.
<code>-analysis_type bc_wc</code>	Best case, worst case analysis. Setup checks use <i>max_condition</i> for clock and data paths. Hold checks use the <i>min_condition</i> for clock and data paths.
<code>-analysis_type on_chip_variation</code>	The min and max operating conditions represent variations on the chip that can occur simultaneously. Setup checks use <i>max_condition</i> for data paths and <i>min_condition</i> for clock paths. Hold checks use <i>min_condition</i> for data paths and <i>max_condition</i> for clock paths. This is the default analysis type.
<code>-library lib</code>	The name of the library that contains <i>condition</i> .
<i>condition</i>	The operating condition for analysis type single.
<code>-min min_condition</code>	The operating condition to use for min paths and hold checks.
<code>-max max_condition</code>	The operating condition to use for max paths and setup checks.
<code>-min_library min_lib</code>	The name of the library that contains <i>min_condition</i> .
<code>-max_library max_lib</code>	The name of the library that contains <i>max_condition</i> .

The `set_operating_conditions` command is used to specify the type of analysis performed and the operating conditions used to derate library data.

set_output_delay	[-rise] [-fall] [-max] [-min] [-clock <i>clock</i>] [-clock_fall] [-reference_pin <i>ref_pin</i>] [-source_latency_included] [-network_latency_included] [-add_delay] <i>delay</i> <i>port_pin_list</i>
-rise	Set the output delay for the rising edge of the input.
-fall	Set the output delay for the falling edge of the input.
-max	Set the maximum output delay.
-min	Set the minimum output delay.
-clock <i>clock</i>	The external check is to <i>clock</i> . The default clock edge is rising.
-clock_fall	The external check is to the falling edge of <i>clock</i> .
-reference_pin <i>ref_pin</i>	The external check is clocked by the clock that arrives at <i>ref_pin</i> .
-add_delay	Add this output delay to any existing output delays.
<i>delay</i>	The external delay to the check clocked by <i>clock</i> .
<i>pin_port_list</i>	A list of pins or ports.

The `set_output_delay` command is used to specify the external delay to a setup/hold check on an output port or internal pin that is clocked by *clock*. Unless the `-add_delay` option is specified any existing output delays are replaced.

The `-reference_pin` option is used to specify a timing check with respect to the arrival on a pin in the clock network. For propagated clocks, the timing check is relative to the clock arrival time at the reference pin (the clock source latency and network latency from the clock source to the reference pin). For ideal clocks, the timing check is relative to the reference pin clock source latency. With the `-clock_fall` flag the timing check is relative to the falling edge of the reference pin. If no clocks arrive at the reference pin the `set_output_delay` command is ignored. If no `-clock` is specified the timing check is with respect to all clocks that arrive at the reference pin. The `-source_latency_included` and `-network_latency_included` options cannot be used with `-reference_pin`.

set_path_margin	[-setup] [-hold] [-rise] [-fall] [-from <i>from_list</i>] [-rise_from <i>from_list</i>] [-fall_from <i>from_list</i>] [-through <i>through_list</i>] [-rise_through <i>through_list</i>] [-fall_through <i>through_list</i>] [-to <i>to_list</i>] [-rise_to <i>to_list</i>] [-fall_to <i>to_list</i>] [-comment <i>comment</i>] <i>margin</i>
-setup	Apply the margin to setup checks.
-hold	Apply the margin to hold checks.
-rise	Apply the margin to rising path edges.
-fall	Apply the margin to falling path edges.
-from <i>from_list</i>	A list of clocks, instances, ports or pins.
-through <i>through_list</i>	A list of instances, pins or nets.
-to <i>to_list</i>	A list of clocks, instances, ports or pins.
-comment <i>comment</i>	Optional comment written by <code>write_sdc</code> .
<i>margin</i>	Signed slack adjustment applied on the capture clock. A positive margin tightens the path and a negative margin loosens it.

The `set_path_margin` command applies a signed slack adjustment to matching timing paths on the capture-clock side. A positive margin makes the path harder to meet and a negative margin makes it easier. If neither `-setup` nor `-hold` is specified the margin applies to both. See `set_false_path` for a description of allowed *from_list*, *through_list* and *to_list* objects. At least one of `-from`, `-through`, or `-to` is required. Matching exceptions are removed with `unset_path_exceptions`.

set_port_fanout_number [-min]
[-max]
fanout
ports

-min Set the min fanout.

-max Set the max fanout.

fanout The external fanout of the ports.

port_list A list of ports.

Set the external fanout for *ports*.

set_power_activity [-global]
[-input]
[-input_ports *ports*]
[-pins *pins*]
[-activity *activity* | -density *density*]
[-duty *duty*]
[-clock *clock*]

-global Set the activity/duty for all non-clock pins.

-input Set the default input port activity/duty.

-input_ports *input_ports* Set the input port activity/duty.

-pins *pins* Set the pin activity/duty.

-activity *activity* The activity, or number of transitions per clock cycle. If *clock* is not specified the clock with the minimum period is used. If no clocks are defined an error is reported.

-density *density* Transitions per library time unit.

-duty *duty* The duty, or probability the signal is high (0 <= duty <= 1.0). Defaults to 0.5.

-clock *clock* The clock to use for the period with -activity. This option is ignored if -density is used.

The `set_power_activity` command is used to set the activity and duty used for power analysis globally or for input ports or pins in the design.

The default input activity for inputs is 0.1 transitions per minimum clock period if a clock is defined or 0.0 if there are no clocks defined. The default input duty is 0.5. This is equivalent to the following command:

```
set_power_activity -input -activity 0.1 -duty 0.5
```

set_propagated_clock *objects*

objects A list of clocks, ports or pins.

The `set_propagated_clock` command changes a clock tree from an ideal network that has no delay one that uses calculated or back-annotated gate and interconnect delays. When *objects* is a port or pin, clock delays downstream of the object are used.

set_pvt [-min]
 [-max]
 [-process *process*]
 [-voltage *voltage*]
 [-temperature *temperature*]
 instances

-min Set the PVT values for max delays.

-max Set the PVT values for min delays.

-process *process* A process value (float).

-voltage *voltage* A voltage value (float).

-temperature *temperature* A temperature value (float).

instances A list instances.

The `set_pvt` command sets the process, voltage and temperature values used during delay calculation for a specific instance in the design.

set_sense	[-type clock data] [-positive] [-negative] [-pulse <i>pulse_type</i>] [-stop_propagation] [-clock <i>clocks</i>] <i>pins</i>
-type clock	Set the sense for clock paths.
-type data	Set the sense for data paths (not supported).
-positive	The clock sense is positive unate.
-negative	The clock sense is negative unate.
-pulse <i>pulse_type</i>	rise_triggered_high_pulse rise_triggered_low_pulse fall_triggered_high_pulse fall_triggered_low_pulse Not supported.
-stop_propagation	Stop propagating <i>clocks</i> at <i>pins</i> .
<i>clocks</i>	A list of clocks to apply the sense.
<i>pins</i>	A list of pins.

The `set_sense` command is used to modify the propagation of a clock signal. The clock sense is set with the `-positive` and `-negative` flags. Use the `-stop_propagation` flag to stop the clock from propagating beyond a pin. The `-positive`, `-negative`, `-stop_propagation`, and `-pulse` options are mutually exclusive. If the `-clock` option is not used the command applies to all clocks that traverse *pins*. The `-pulse` option is currently not supported.

set_timing_derate	[-rise] [-fall] [-early] [-late] [-clock] [-data] [-net_delay] [-cell_delay] [-cell_check] <i>derate</i> <i>objects</i>
-rise	Set the derating for rising delays.
-fall	Set the derating for falling delays.
-early	Derate early (min) paths.
-late	Derate late (max) paths.
-clock	Derate paths in the clock network.
-data	Derate data paths.
-net_delay	Derate net (interconnect) delays.
-cell_delay	Derate cell delays.
-cell_check	Derate cell timing check margins.
<i>derate</i>	The derating factor to apply to delays.
<i>objects</i>	A list of instances, library cells, or nets.

The `set_timing_derate` command is used to derate delay calculation results used by the STA. If the `-early` and `-late` flags are omitted the both min and max paths are derated. If the `-clock` and `-data` flags are not used the derating both clock and data paths are derated.

Use the `unset_timing_derate` command to remove all derating factors.

set_resistance	[-max] [-min] <i>resistance</i> <i>nets</i>
-----------------------	--

<code>-min</code>	The resistance for minimum path delay calculation.
<code>-max</code>	The resistance for maximum path delay calculation.
<code>resistance</code>	The net resistance.
<code>nets</code>	A list of nets.

set_units `[-capacitance cap_unit]`
 `[-resistance res_unit]`
 `[-time time_unit]`
 `[-voltage voltage_unit]`
 `[-current current_unit]`
 `[-power power_unit]`
 `[-distance distance_unit]`

<code>-capacitance <i>cap_unit</i></code>	The capacitance scale factor followed by 'f'.
<code>-resistance <i>res_unit</i></code>	The resistance scale factor followed by 'ohm'.
<code>-time <i>time_unit</i></code>	The time scale factor followed by 's'.
<code>-voltage <i>voltage_unit</i></code>	The voltage scale factor followed by 'v'.
<code>-current <i>current_unit</i></code>	The current scale factor followed by 'A'.
<code>-power <i>power_unit</i></code>	The power scale factor followed by 'w'.

The `set_units` command is used to **check** the units used by the STA command interpreter when parsing commands and reporting results. If the current units differ from the `set_unit` value a warning is printed. Use the `set_cmd_units` command to change the command units.

Units are specified as a scale factor followed by a unit name. The scale factors are as follows.

```

M 1E+6
k 1E+3
m 1E-3
u 1E-6
n 1E-9
p 1E-12
f 1E-15

```

An example of the `set_units` command is shown below.

set_units -time ns -capacitance pF -current mA -voltage V -resistance kOhm

set_wire_load_min_block_size *size*

The set_wire_load_min_block_size command is not supported.

set_wire_load_mode top|enclosed|segmented

top

enclosed

segmented

The set_wire_load_mode command is ignored during timing but is included in SDC files that are written.

set_wire_load_model -name *model_name*
[-library *library*]
[-max]
[-min]
[*objects*]

-name *model_name* The name of a wire load model.

-library *library* Library to look for *model_name*.

-max The wire load model is for maximum path delays.

-min The wire load model is for minimum path delays.

objects Not supported.

set_wire_load_selection_group [-library *library*]
[-max]
[-min]
group_name
[*objects*]

library Library to look for *group_name*.

<code>-max</code>	The wire load selection is for maximum path delays.
<code>-min</code>	The wire load selection is for minimum path delays.
<code>group_name</code>	A wire load selection group name.
<code>objects</code>	Not supported.

The `set_wire_load_selection_group` command is parsed but not supported.

<code>suppress_msg</code>	<code>msg_ids</code>
<code>msg_ids</code>	A list of error/warning message IDs to suppress.

The `suppress_msg` command suppresses specified error/warning messages by ID. The list of message IDs can be found in `doc/messages.txt`.

<code>unset_case_analysis</code>	<code>port_or_pin_list</code>
<code>port_or_pin_list</code>	A list of ports or pins.

The `unset_case_analysis` command removes the constant values defined by the `set_case_analysis` command.

<code>unset_clock_latency</code>	<code>[-source]</code> <code>objects</code>
---	--

<code>-source</code>	Specifies source clock latency (clock insertion delay).
<code>objects</code>	A list of clocks, pins or ports.

The `unset_clock_latency` command removes the clock latency set with the `set_clock_latency` command.

<code>unset_clock_transition</code>	<code>clocks</code>
<code>clocks</code>	A list of clocks.

The `unset_clock_transition` command removes the clock transition set with the `set_clock_transition` command.

unset_clock_uncertainty [-from|-rise_from|-fall_from *from_clock*]
[-to|-rise_to|-fall_to *to_clock*]
[-rise]
[-fall]
[-setup]
[-hold]
[*objects*]

-from *from_clock*

-to *to_clock*

-rise The uncertainty is for the rising edge of the clock.

-fall The uncertainty is for the falling edge of the clock.

-setup *uncertainty* is the setup check uncertainty.

-hold *uncertainty* is the hold uncertainty.

uncertainty Clock uncertainty.

objects A list of clocks, ports or pins.

The `unset_clock_uncertainty` command removes clock uncertainty defined with the `set_clock_uncertainty` command.

unset_data_check [-from|-rise_from|-fall_from *from_object*]
[-to|-rise_to|-fall_to *to_object*]
[-setup]
[-hold]
[-clock *clock*]

-from *from_object* A pin used as the timing check reference.

-to *to_object* A pin that the setup/hold check is applied to.

-setup Add a setup timing check.

-hold Add a hold timing check.

clock The setup/hold check clock.

The `unset_clock_transition` command removes a setup or hold check defined by the `set_data_check` command.

`unset_disable_inferred_clock_gating` *objects*

<i>objects</i>	A list of clock gating instances, clock gating pins, or clock enable pins.
----------------	--

The `unset_disable_inferred_clock_gating` command removes a previous `set_disable_inferred_clock_gating` command.

```
unset_disable_timing    [-from from_port]  
                          [-to to_port]  
                          objects
```

from_port

to_port

<i>objects</i>	A list of instances, ports, pins, cells or [library/]cell/port.
----------------	---

The `unset_disable_timing` command is used to remove the effect of previous `set_disable_timing` commands.

```
unset_input_delay    [-rise]
                    [-fall]
                    [-max]
                    [-min]
                    [-clock clock]
                    [-clock_fall]
                    port_pin_list
```

```
-rise      Unset the arrival time for the rising edge of the input.
```

-fall	Unset the arrival time for the falling edge of the input.
-------	---

-max	Unset the minimum arrival time.
------	---------------------------------

```
-min          Unset the maximum arrival time.
```

clock Unset the arrival time from *clock*.

<code>-clock_fall</code>	Unset the arrival time from the falling edge of <i>clock</i>
--------------------------	--

pin_port_list A list of pins or ports.

The `unset_input_delay` command removes a previously defined `set_input_delay`.

unset_output_delay	<code>[-rise]</code> <code>[-fall]</code> <code>[-max]</code> <code>[-min]</code> <code>[-clock <i>clock</i>]</code> <code>[-clock_fall]</code> <i>port_pin_list</i>
<code>-rise</code>	This is the arrival time for the rising edge of the input.
<code>-fall</code>	This is the arrival time for the falling edge of the input.
<code>-max</code>	This is the minimum arrival time.
<code>-min</code>	This is the maximum arrival time.
<i>clock</i>	The arrival time is from this clock.
<code>-clock_fall</code>	The arrival time is from the falling edge of <i>clock</i>
<i>pin_port_list</i>	A list of pins or ports.

The `unset_output_delay` command a previously defined `set_output_delay`.

unset_path_exceptions	<code>[-setup]</code> <code>[-hold]</code> <code>[-rise]</code> <code>[-fall]</code> <code>[-from -rise_from -fall_from <i>from</i>]</code> <code>[-through -rise_through -fall_through <i>through</i>]</code> <code>[-to -rise_to -fall_to <i>to</i>]</code>
<code>-setup</code>	Unset path exceptions for setup checks.
<code>-hold</code>	Unset path exceptions for hold checks.
<code>-rise</code>	Unset path exceptions for rising path edges.

<code>-fall</code>	Unset path exceptions for falling path edges.
<code>-from <i>from</i></code>	A list of clocks, instances, ports or pins.
<code>-through <i>through</i></code>	A list of instances, pins or nets.
<code>-to <i>to</i></code>	A list of clocks, instances, ports or pins.

The `unset_path_exceptions` command removes any matching `set_false_path`, `set_multicycle_path`, `set_max_delay`, `set_min_delay`, and `set_path_margin` exceptions.

<code>unset_power_activity</code>	<code>[-global]</code> <code>[-input]</code> <code>[-input_ports <i>ports</i>]</code> <code>[-pins <i>pins</i>]</code>
<code>-global</code>	Set the activity/duty for all non-clock pins.
<code>-input</code>	Set the default input port activity/duty.
<code>-input_ports <i>input_ports</i></code>	Set the input port activity/duty.
<code>-pins <i>pins</i></code>	Set the pin activity/duty.
<code>-activity <i>activity</i></code>	The activity, or number of transitions per clock cycle. If <i>clock</i> is not specified the clock with the minimum period is used. If no clocks are defined an error is reported.

The `unset_power_activity_command` is used to undo the effects of the `set_power_activity` command.

<code>unset_propagated_clock</code>	<i>objects</i>
<i>objects</i>	A list of clocks, ports or pins.

Remove a previous `set_propagated_clock` command.

`unset_timing_derate`

Remove all derating factors set with the `set_timing_derate` command.

unsuppress_msg	<i>msg_ids</i>
-----------------------	----------------

<i>msg_ids</i>	A list of error/warning message IDs to unsuppress.
----------------	--

The `unsuppress_msg` command removes suppressions for the specified error/warning messages by ID. The list of message IDs can be found in *doc/messages.txt*.

user_run_time	
----------------------	--

Returns the total user cpu run time in seconds as a float.

with_output_to_variable	<i>var</i> { <i>commands</i> }
--------------------------------	--------------------------------

<i>var</i>	The name of a variable to save the output of <i>commands</i> to.
------------	--

<i>commands</i>	TCL commands that the output will be redirected from.
-----------------	---

The `with_output_to_variable` command redirects the output of TCL commands to a variable.

write_path_spice	<code>-path_args</code> <i>path_args</i> <code>-spice_file</code> <i>spice_file</i> <code>-lib_subckt_file</code> <i>lib_subckts_file</i> <code>-model_file</code> <i>model_file</i> <code>-power</code> <i>power</i> <code>-ground</code> <i>ground</i> <code>[-simulator</code> <i>hspice</i> <i>ngspice</i> <i>xyce</i>] <i>path_args</i> <code>-from</code> <code>-through</code> <code>-to</code> arguments as in <code>report_checks</code> .
-------------------------	---

<i>spice_file</i>	Directory and path prefix for spice output files.
-------------------	---

<i>lib_subckts_file</i>	Cell transistor level subckts.
-------------------------	--------------------------------

<i>model_file</i>	Transistor model definitions .included by <i>spice_file</i> .
-------------------	---

<i>power</i>	Voltage supply name in <i>voltage_map</i> of the default liberty library.
--------------	---

<i>ground</i>	Ground supply name in <i>voltage_map</i> of the default liberty library.
---------------	--

<code>-simulator</code>	Simulator that will read the spice netlist.
-------------------------	---

The `write_path_spice` command writes a spice netlist for timing paths. Use *path_args* to specify `-from/-through/-to` as arguments to the `find_timing_paths` command. For each path, a spice netlist and the subckts referenced by the path are written in *spice_directory*. The spice netlist is written in *path_<id>.sp* and subckt file is *path_<id>.subckt*.

The spice netlists used by the path are written to *subckt_file*, which *spice_file* `.includes`. The device models used by the spice subckt netlists in *model_file* are also `.included` in *spice_file*. Power and ground names are specified with the `-power` and `-ground` arguments. The spice netlist includes a piecewise linear voltage source at the input and `.measure` statement for each gate delay and pin slew.

Example command:

```
write_path_spice -path_args {-from "in0" -to "out1" -unconstrained} \
  -spice_directory $result_dir \
  -lib_subckt_file "write_spice1.subckt" \
  -model_file "write_spice1.models" \
  -power VDD -ground VSS
```

When the simulator is hspice, `.measure` statements will be added to the spice netlist.

When the simulator is Xyce, the `.print` statement selects the CSV format and writes the waveform data to a file name `path_<id>.csv` so the results can be used by gnuplot.

write_sdc	<code>[-digits <i>digits</i>]</code> <code>[-gzip]</code> <code>[-no_timestamp]</code> <i>filename</i>
<i>digits</i>	The number of digits after the decimal point to report. The default is 4.
<code>-gzip</code>	Compress the SDC with gzip.
<code>-no_timestamp</code>	Do not include a time and date in the SDC file.
<i>filename</i>	The name of the file to write the constraints to.

Write the constraints for the design in SDC format to *filename*.

write_sdf	<code>[-scene <i>scene</i>]</code> <code>[-divider / .]</code> <code>[-include_typ]</code> <code>[-digits <i>digits</i>]</code> <code>[-gzip]</code> <code>[-no_timestamp]</code> <code>[-no_version]</code> <i>filename</i>
<i>scene</i>	Write delays for <i>scene</i> .

<code>-divider</code>	Divider to use between hierarchy levels in pin and instance names.
<code>-include_typ</code>	Include a 'typ' value in the SDF triple that is the average of min and max delays to satisfy some Verilog simulators that require three values in the delay triples.
<code>-digits <i>digits</i></code>	The number of digits after the decimal point to report. The default is 4.
<code>-gzip</code>	Compress the SDF using gzip.
<code>-no_timestamp</code>	Do not write a DATE statement.
<code>-no_version</code>	Do not write a VERSION statement.
<i>filename</i>	The SDF filename to write.

Write the delay calculation delays for the design in SDF format to *filename*. If `-corner` is not specified the min/max delays are across all corners. With `-corner` the min/max delays for *corner* are written. The SDF TIMESCALE is same as the `time_unit` in the first liberty file read.

<code>write_timing_model</code>	<code>[-library_name <i>lib_name</i>]</code> <code>[-cell_name <i>cell_name</i>]</code> <code>[-scene <i>scene</i>]</code> <i>filename</i>
<i>lib_name</i>	The name to use for the liberty library. Defaults to <i>cell_name</i> .
<i>cell_name</i>	The name to use for the liberty cell. Defaults to the top level module name.
<i>scene</i>	The scene to use for extracting the model.
<i>filename</i>	Filename for the liberty timing model.

The `write_timing_model` command constructs a liberty timing model for the current design and writes it to *filename*. *cell_name* defaults to the cell name of the top level block in the design.

The SDC used to extract the block should include the clock definitions. If the block contains a clock network `set_propagated_clock` should be used so the clock delays are included in the timing model. The following SDC commands are ignored when building the timing model.

```
set_input_delay
set_output_delay
set_load
set_timing_derate
```

Using `set_input_transition` with the slew from the block context will be used will improve the match between the timing model and the block netlist. Paths defined on clocks that are defined on internal pins are ignored because the model has no way to include the clock definition.

The resulting timing model can be used in a hierarchical timing flow as a replacement for the block to speed up timing analysis. This hierarchical timing methodology does not handle timing exceptions that originate or terminate inside the block. The timing model includes:

```
combinational paths between inputs and outputs
setup and hold timing constraints on inputs
clock to output timing paths
```

Resistance of long wires on inputs and outputs of the block cannot be modeled in Liberty. To reduce inaccuracies from wire resistance in technologies with resistive wires place buffers on inputs and outputs.

The extracted timing model setup/hold checks are scalar (no input slew dependence). Delay timing arcs are load dependent but do not include input slew dependency.

write_verilog	<code>[-include_pwr_gnd]</code> <code>[-remove_cells lib_cells]</code> <i>filename</i>
<code>-include_pwr_gnd</code>	Include power and ground pins on instances.
<code>-remove_cells lib_cells</code>	Liberty cells to remove from the Verilog netlist. Use <code>get_lib_cells</code> , a list of cells names, or a cell name with wildcards.
<i>filename</i>	Filename for the liberty library.

The `write_verilog` command writes a Verilog netlist to *filename*. Use `-sort` to sort the instances so the results are reproducible across operating systems. Use `-remove_cells` to remove instances of *lib_cells* from the netlist.

Filter Expressions

The `get_cells`, `get_pins`, `get_ports` and `get_timing_edges` functions support filtering the returned objects by property values. Supported filter expressions are shown below.

<i>property</i>	Return objects with <i>property</i> value equal to 1.
<i>property==value</i>	Return objects with <i>property</i> value equal to <i>value</i> .

<i>property</i> =~ <i>pattern</i>	Return objects with <i>property</i> value that matches <i>pattern</i> .
<i>property</i> != <i>value</i>	Return objects with <i>property</i> value not equal to <i>value</i> .
<i>property</i> !~ <i>value</i>	Return objects with <i>property</i> value that does not match <i>pattern</i> .
<i>expr1</i> && <i>expr2</i>	Return objects with <i>expr1</i> and <i>expr2</i> . <i>expr1</i> and <i>expr2</i> are one of the first three property value forms shown above.
<i>expr1</i> <i>expr2</i>	Return objects with <i>expr1</i> or <i>expr2</i> . <i>expr1</i> and <i>expr2</i> are one of the first three property value forms shown above.

Where *property* is a property supported by the `get_property` command. Note that if there are spaces in the expression it must be enclosed in quotes so that it is a single argument.

Variables

hierarchy_separator	Any character.
----------------------------	----------------

The `hierarchy_separator` separates instance names in a hierarchical instance, net, or pin name. The default value is `'/'`.

sta_continue_on_error	0 1
------------------------------	-------

The `include` and `read_sdc` commands stop and report any errors encountered while reading a file unless `sta_continue_on_error` is 1. The default value is 0.

sta_crpr_mode	same_pin same_transition
----------------------	----------------------------

When the data and clock paths of a timing check overlap (see `sta_crpr_enabled`), pessimism is removed independent of whether of the path rise/fall transitions. When `sta_crpr_mode` is `same_transition`, the pessimism is only removed if the path rise/fall transitions are the same. The default value is `same_pin`.

sta_cond_default_arcs_enabled	0 1
--------------------------------------	-------

When set to 0, default timing arcs with no condition (Liberty timing arcs with no “when” expression) are disabled if there are other conditional timing arcs between the same pins. The default value is 1.

sta_crpr_enabled	0 1
-------------------------	-------

During min/max timing analysis for `on_chip_variation` the data and clock paths may overlap. For a setup check the maximum path delays are used for the data and the minimum path delays are used for the clock. Because the gates cannot simultaneously have minimum and maximum delays the timing check slack is pessimistic. This

pessimism is known as Common Reconvergent Pessimism Removal, or “CRPR”. Enabling CRPR slows down the analysis. The default value is 1.

sta_dynamic_loop_breaking	0 1
----------------------------------	-------

When `sta_dynamic_loop_breaking` is 0, combinational logic loops are disabled by disabling a timing arc that closes the loop. When `sta_dynamic_loop_breaking` is 1, all paths around the loop are reported. The default value is 0.

sta_gated_clock_checks_enabled	0 1
---------------------------------------	-------

When `sta_gated_clock_checks_enabled` is 1, clock gating setup and hold timing checks are checked. The default value is 1.

sta_input_port_default_clock	0 1
-------------------------------------	-------

When `sta_input_port_default_clock` is 1 a default input arrival is added for input ports that do not have an arrival time specified with the `set_input_delay` command. The default value is 0.

sta_internal_bidirect_instance_paths_enabled	0 1
---	-------

When set to 0, paths from bidirectional (inout) ports back into the instance are disabled. When set to 1, paths from bidirectional ports back into the instance are enabled. The default value is 0.

sta_pocv_mode	scalar normal skew_normal
----------------------	-------------------------------

Enable parametric on chip variation using statistical timing analysis. The default value is `scalar`.

sta_pocv_quartile	quartile
--------------------------	----------

The target quantile of a delay probability distribution (confidence level). The default value is 3 standard deviations, or sigma.

sta_propagate_all_clocks	0 1
---------------------------------	-------

All clocks defined after `sta_propagate_all_clocks` is set to 1 are propagated. If it is set before any clocks are defined it has the same effect as

set_propagated_clock [all_clocks]

After all clocks have been defined. The default value is 0.

sta_propagate_gated_clock_enable	0 1
---	-------

When set to 1, paths of gated clock enables are propagated through the clock gating instances. If the gated clock controls sequential elements setting `sta_propagate_gated_clock_enable` to 0 prevents spurious paths from the clock enable. The default value is 1.

sta_recovery_removal_checks_enabled	0 1
--	-------

When `sta_recovery_removal_checks_enabled` is 0, recovery and removal timing checks are disabled. The default value is 1.

sta_report_default_digits	integer
----------------------------------	---------

The number of digits to print after a decimal point. The default value is 2.

sta_preset_clear_arcs_enabled	0 1
--------------------------------------	-------

When set to 1, paths through asynchronous preset and clear timing arcs are searched. The default value is 0.

Alphabetical Index

all_clocks.....	7
all_inputs.....	7
all_outputs.....	8
all_registers.....	8
check_setup.....	9
Command Line Arguments.....	1
Commands.....	7
connect_pin.....	9
create_generated_clock.....	11
create_voltage_area.....	12
current_design.....	12
current_instance.....	13
define_scene.....	13
delete_clock.....	13
delete_from_list.....	13
delete_generated_clock.....	14
delete_instance.....	14
delete_net.....	14
disconnect_pin.....	14
elapsed_run_time.....	14
Example Command Scripts.....	1
Filter Expressions.....	84
find_timing_paths.....	15
get_cells.....	17
get_clocks.....	17
get_fanin.....	18
get_fanout.....	19
get_full_name.....	19
get_lib_pins.....	20
get_libs.....	21
get_name.....	22
get_nets.....	22
get_pins.....	23
get_ports.....	23
get_property.....	24
get_scenes.....	28
get_timing_edges.....	28
group_path.....	29
hierarchy_separator.....	85
include.....	30
link_design.....	30
make_instance.....	30
make_net.....	31
Power Analysis.....	3
read_liberty.....	31
read_saif.....	32
read_sdc.....	33
read_sdf.....	33
read_spf.....	34
read_vcd.....	35

read_verilog.....	35
redirection.....	5
replace_activity_annotation.....	36
replace_cell.....	35
report_annotated_check.....	36
report_annotated_delay.....	37
report_check_types.....	41
report_checks.....	38
report_clock_latency.....	42
report_clock_min_period.....	42
report_clock_properties.....	43
report_clock_skew.....	43
report_dcalc.....	43
report_disabled_edges.....	44
report_edges.....	44
report_instance.....	44
report_lib_cell.....	44
report_net.....	45
report_parasitic_annotation.....	45
report_power.....	45
report_slews.....	46
report_tns.....	46
report_units.....	46
report_wns.....	47
report_worst_slack.....	47
set_assigned_check.....	48
set_assigned_delay.....	49
set_assigned_transition.....	49
set_case_analysis.....	50
set_clock_gating_check.....	50
set_clock_groups.....	51
set_clock_latency.....	52
set_clock_transition.....	52
set_clock_uncertainty.....	53
set_cmd_units.....	54
set_data_check.....	55
set_disable_inferred_clock_gating.....	55
set_disable_timing.....	55
set_drive.....	56
set_driving_cell.....	57
set_false_path.....	58
set_fanout_load.....	59
set_hierarchy_separator.....	59
set_ideal_latency.....	59
set_ideal_network.....	59
set_ideal_transition.....	59
set_input_delay.....	59
set_input_transition.....	61
set_level_shifter_strategy.....	61
set_level_shifter_threshold.....	61
set_load.....	61
set_logic_dc.....	62
set_logic_one.....	62
set_logic_zero.....	63

set_max_area.....	63
set_max_capacitance.....	63
set_max_delay.....	63
set_max_dynamic_power.....	64
set_max_fanout.....	64
set_max_leakage_power.....	64
set_max_time_borrow.....	64
set_max_transition.....	65
set_min_capacitance.....	65
set_min_delay.....	66
set_min_pulse_width.....	67
set_mode.....	67
set_multicycle_path.....	67
set_operating_conditions.....	68
set_output_delay.....	69
set_path_margin.....	70
set_port_fanout_number.....	70
set_power_activity.....	70
set_propagated_clock.....	71
set_pvt.....	71
set_resistance.....	73
set_sense.....	72
set_timing_derate.....	73
set_units.....	74
set_wire_load_min_block_size.....	75
set_wire_load_mode.....	75
set_wire_load_model.....	75
set_wire_load_selection_group.....	75
SPEF.....	34
sta_cond_default_arcs_enabled.....	85
sta_continue_on_error.....	85
sta_crpr_enabled.....	85
sta_crpr_mode.....	85
sta_dynamic_loop_breaking.....	85
sta_gated_clock_checks_enabled.....	85
sta_input_port_default_clock.....	86
sta_internal_bidirect_instance_paths_enabled.....	86
sta_pocv_enabled.....	86
sta_preset_clear_arcs_enabled.....	87
sta_propagate_all_clocks.....	86
sta_propagate_gated_clock_enable.....	86
sta_recovery_removal_checks_enabled.....	86
sta_report_default_digits.....	86
suppress_msg.....	76
TCL Interpreter.....	5
Timing Analysis using SDF.....	2
Timing Analysis with Multiple Corners and Modes.....	3
Timing Analysis with Multiple Process Corners.....	2
unset_case_analysis.....	76
unset_clock_latency.....	76
unset_clock_transition.....	76
unset_clock_uncertainty.....	77
unset_data_check.....	77
unset_disable_inferred_clock_gating.....	78

unset_disable_timing.....	78
unset_input_delay.....	78
unset_output_delay.....	79
unset_path_exceptions.....	79
unset_power_activity.....	80
unset_propagated_clock.....	80
unset_timing_derate.....	80
unsuppress_msg.....	81
user_run_time.....	81
Variables.....	85
verilog netlist.....	35
with_output_to_variable.....	81
write_path_spice.....	81
write_sdc.....	82
write_sdf.....	82
write_timing_model.....	83
write_verilog.....	84

Version 3.0.0, Mar 7, 2026

Copyright (c) 2026, Parallax Software, Inc.

This program is free software: you can redistribute it and/or modify it under the terms of the GNU General Public License as published by the Free Software Foundation, either version 3 of the License, or (at your option) any later version.

This program is distributed in the hope that it will be useful, but WITHOUT ANY WARRANTY; without even the implied warranty of MERCHANTABILITY or FITNESS FOR A PARTICULAR PURPOSE. See the GNU General Public License for more details.

You should have received a copy of the GNU General Public License along with this program. If not, see <https://www.gnu.org/licenses/>.